

SilverStream eXtend Composer™

JDBC Connect

User's Guide

Version 4.0

June 2002

Copyright © 2000-2002 SilverStream Software, Inc. All rights reserved.

SilverStream software products are copyrighted and all rights are reserved by SilverStream Software, Inc. This documentation is also copyrighted and all rights are reserved. This documentation may not, in whole or in part, be copied, photocopied, translated, or reduced to any electronic medium or machine-readable form without prior consent, in writing, from SilverStream Software, Inc.

The information in this documentation is subject to change without notice, and SilverStream Software assumes no responsibility for any errors that may appear here.

SilverStream is a registered trademark, and SilverStream Composer , JBroker, and eXtend are trademarks of SilverStream Software, Inc.

Sun Microsystems, Inc. Sun, the Sun logo, Sun Microsystems, JavaBeans, Enterprise JavaBeans, JavaServer Pages, Java Naming and Directory Interface, JDK, JDBC, Java, HotJava, HotJava Views, Visual Java, Solaris, Sun Workstation, Java WorkShop, the Java Coffee Cup logo, and Visual Java are trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries.

Sybase, Inc. Sybase Adaptive Server Anywhere is licensed from Sybase Inc. Copyright Sybase Inc. All rights reserved.

All other trademarks or registered trademarks are properties of their respective owners. Companies, names, and/or data used in screens and sample output are fictitious unless otherwise noted.

Contents

Preface v

- Purpose v
- Audience v
- Organization v
- Conventions v

Chapter 1 Welcome to eXtend Composer and JDBC 7

- Before You Begin 7
- About eXtend Connects 7
- What is JDBC? 8
- What Does JDBC Do? 9
- About eXtend's JDBC Component 10
- What Applications Can You Build Using the JDBC Component Editor? 11
- Where to Go for More Help 11

Chapter 2 Getting Started with the JDBC Component Editor 13

- Creating a JDBC Connection Resource 13
 - About Constant and Expression Driven Connection Parameters 13
 - About JDBC Drivers and Connection Pools 14
- Creating XML Templates for Your Component 18

Chapter 3 Creating a JDBC Component 19

- Before Creating a JDBC Component 19
- About the JDBC Component Editor Window 22
 - About the Query Pane 22

Chapter 4 Performing JDBC Actions 25

- About Actions 25
- The SQL Statement Action 25
 - Prepared Statements 25
- Building an SQL Statement 27
 - Building an Example Query 28
 - Using Stored Procedures 29
 - Colons in SQL Statements 33
 - Executing the SQL Statement 33

Checking the Results	34
Mapping Results into the Output DOM	35
The SQL Batch Action	36
Start Batch	37
Execute Batch	37
Discard Batch	37
Creating Batch actions	38
JDBC-Specific Expression Builder Properties	39
Using Other Actions in the JDBC Component Editor	40
Handling Errors and SQL Messages	44

Chapter 5 Using Custom Result Mapping 45

About Default Result Mapping	45
About Custom Result Mapping	46
About Custom Result Mapping and Aliases	47
Using the MapTarget Tab	47
Looking at a MapTarget Example	50
Using The Detail Rows Tab	52
Looking at a Detail Rows Example	53
Using the Declare Group/Repeat Tab	56
Looking at a Declare Group/Repeat Example	57

Chapter 6 Stored Procedure Mapping 63

About Stored Procedure Mapping	63
Binding Rules	64
Using the Stored Procedure Mapping Setup Dialog	64
Returned Result Set	66

Appendix A JDBC Glossary 67

Appendix B Reserved Words 69

Preface

Purpose

This guide describes how to use the eXtend Connect, referred to as the JDBC Component Editor. The JDBC Component Editor is a standard component editor in eXtend Composer.

Audience

To be successful in using the JDBC Component Editor, you should be familiar with using eXtend Composer. Practicing with the XML Map Component Editor will help you use the JDBC Component Editor. To create JDBC components for your XML integration application, you must also have an understanding of Structured Query Language (SQL) techniques and results.

Organization

This guide is organized as follows:

Chapter 1, *Welcome to eXtend Composer and JDBC*, gives a definition and overview of the JDBC Component Editor.

Chapter 2, *Getting Started with the JDBC Component Editor*, describes the necessary preparations for creating a JDBC component.

Chapter 3, *Creating a JDBC Component*, describes the parts of the component editor.

Chapter 4, *Performing JDBC Actions*, describes how to use the SQL Statement action and the SQL Batch action as well as Prepared Statements.

Chapter 5, *Using Custom Result Mapping*, describes when and how to use custom mapping.

Conventions

This guide uses the following typographical conventions.

Bold typeface within instructions indicate action items, including:

- Menu selections
- Form selections
- Dialog box items

Italic typeface indicates:

- Variable information that you supply
- Technical terms used for the first time
- Titles of other SilverStream publications

Monospaced typeface indicates:

- Method names
- Code examples
- System input
- Operating system objects

Bold sans-serif typeface indicates:

- Uniform Resource Identifiers
- Database driver names
- File names

1 Welcome to eXtend Composer and JDBC

Before You Begin

Welcome to the *SilverStream eXtend JDBC Connect User's Guide*. This Guide is a companion to the *eXtend Composer User's Guide*, which details how to use all the features of Composer except for the Connect Component Editors. So, if you haven't looked at the *Composer User's Guide* yet, please familiarize yourself with it before using this Guide.

eXtend Composer provides separate Component Editors for each Connect, such as the JDBC connector. The special features of each component editor are described in separate Guides like this one.

If you have been using eXtend Composer, and are familiar with the core component editor (the XML Map Component Editor), then this Guide should get you started with the JDBC Component Editor.

NOTE To be successful with this Component Editor, you must be familiar with writing and constructing SQL statements.

About eXtend Connects

eXtend is built upon a simple hub and spoke architecture. The hub is a robust XML transformation engine that accepts XML documents, processes the documents, and returns an XML document. The spokes or Connects are plug-in modules that "XML enable" sources of data that are not XML-aware, bringing their data into the hub for processing as XML. These data sources can be anything from legacy COBOL / VSAM managed information to Message Queues to HTML pages. eXtend Connects can be categorized by the integration strategy each one employs to XML enable an information source. The integration strategies are a reflection of the major divisions used in modern systems designs for Internet based computing architectures. Depending on your B2Bi needs, eXtend can integrate your business systems at the User Interface, Program Logic, and/or Data levels. (See Figure 1-1.)

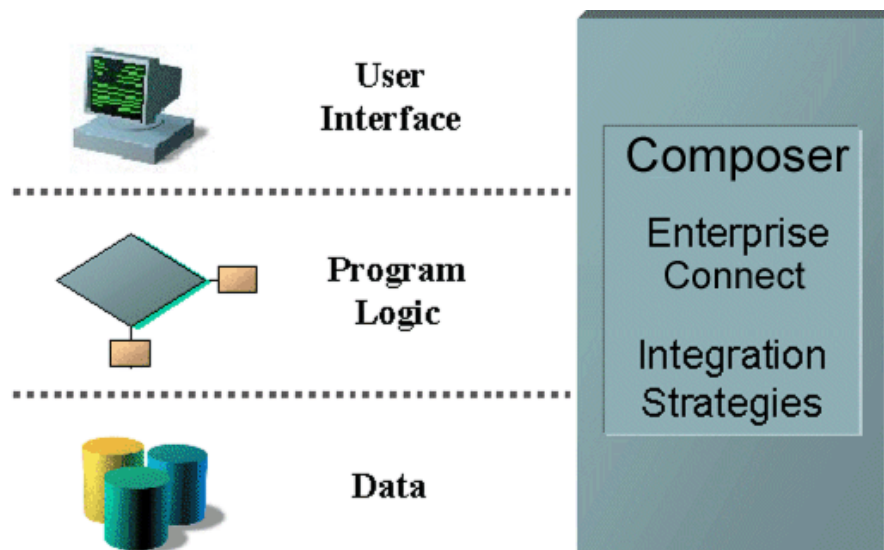
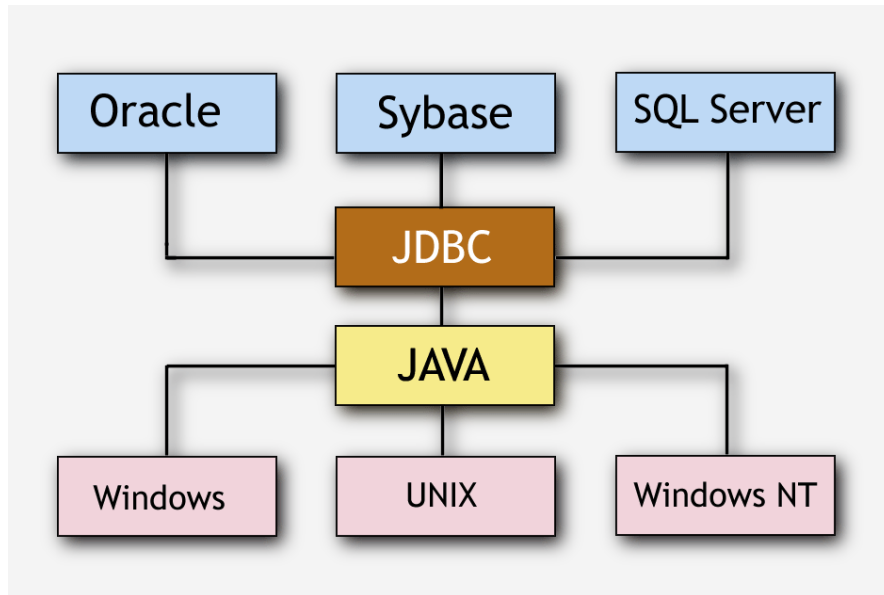


Figure 1-1

What is JDBC?

JDBC is a Java-based API (Application Programming Interface) for executing SQL statements. While often mistaken as an acronym meaning “Java Database Connectivity,” JDBC is in fact not an acronym at all, but a trademarked name. JDBC consists of a set of classes and interfaces written in the Java programming language that allows you to write one program to access different databases such as Oracle, Sybase, Informix, etc., rather than needing to write a separate program for each one.

You can write a single program using the JDBC API and the program is able to send SQL statements to the appropriate database. And since the application is written in the Java programming language, there is no need to write different applications to run on different platforms. The combination of Java and JDBC lets you write it once and run it anywhere, as the following illustration shows.

*Figure 1-2*

What Does JDBC Do?

JDBC makes it possible to do the following:

- Establish a connection with a database
- Send SQL statements (or queries) to be processed by the database
- Process the results of the database processing

JDBC is a low-level interface used to call SQL commands directly. It is integrated into Composer to interface between components and databases, allowing the program to establish connections with the databases, send the SQL statements, and process the results. Composer provides tools that enable visual construction of the necessary SQL commands.

About eXtend's JDBC Component

Much like the XML Map Component, the JDBC Component is designed to map, transform, and transfer data between two different XML templates (i.e., request and response XML documents). However, it is specialized to make a connection to a database, process SQL statements against the database using elements from a DOM within the query, and then map the results of the query to a DOM.

A JDBC Component can perform simple data manipulations, such as mapping and transferring data from one XML document to another, or from an XML document to a database table. It can also perform sophisticated manipulations, such as requesting data from disparate databases, transforming data from and to one or more documents, executing SQL transactions against the database, and even transforming the documents themselves. Like an XML Map Component, the JDBC Component can process XSL, send mail, and post and receive XML documents using the HTTP protocol.

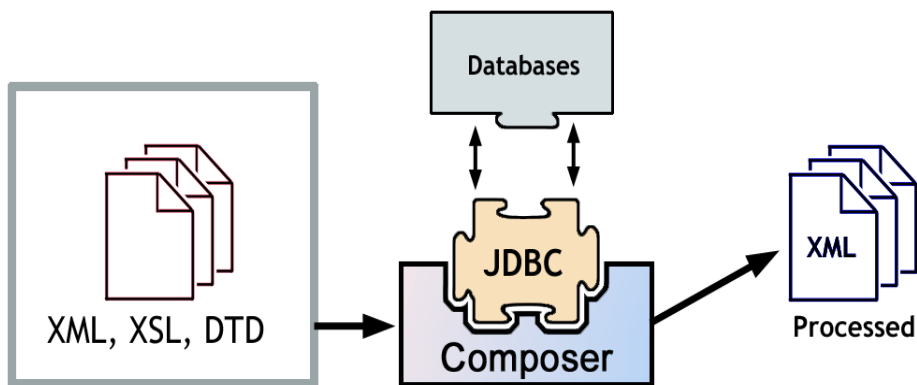


Figure 1-3: The JDBC Connect uses eXtend Composer as the backplane for XML-based data interactions, making it possible to reach into databases at runtime (and design time). Using eXtend Composer, you can assemble Action Models within a JDBC Component to carry out sophisticated data transformations, using HTTP (optionally) as a transport mechanism. Live database connections are available at design time, so that you can edit and debug SQL queries as part of the design process.

What Applications Can You Build Using the JDBC Component Editor?

You can build any business-to-business application that needs to push data into or pull data from a JDBC-accessible data store and uses XML as the interchange format. For example, you can write an application that retrieves the description, picture and price of a product from a database and displays it in the user's browser. If the information resides in two or more databases, you can merge the information from separate databases before displaying it to the user.

Where to Go for More Help

Perhaps the best way to understand Composer is to see it in action. Composer includes a fully functional project, referred to as the Tutorial, that you can step through to see how the application handles a practical business operation. The project file is **Tutorial.xcp** and is located in the **\Samples\Tutorial** folder, under eXtend.

Additionally, you may find valuable information on the SilverStream website:
b2b.silverstream.com.

2 Getting Started with the JDBC Component Editor

Creating a JDBC Connection Resource

Before you create a JDBC Component, you will find it necessary to create a Connection Resource to access the SQL database. Each Connect, including the JDBC connector, uses its own Connection type. Each Connection type is differentiated by the number and types of parameters used to connect to the specific external data source.

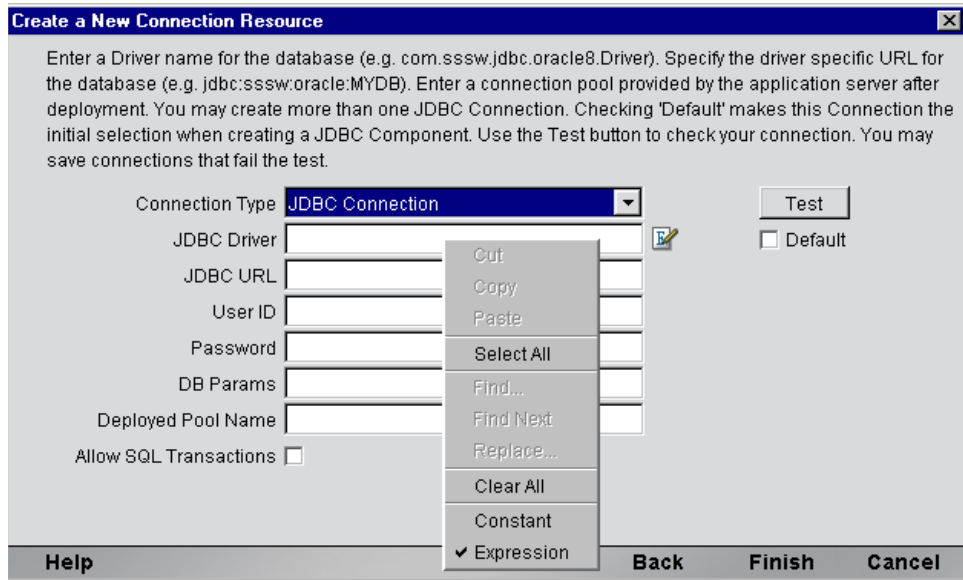
About Constant and Expression Driven Connection Parameters

You can specify Connection parameter values in one of two ways: as Constants or as Expressions. A constant based parameter uses the value you type in the Connection dialog every time the Connection is used. An expression based parameter allows you to set the value using a programmatic expression, which can result in a different value each time the connection is used at runtime. This allows the Connection's behavior to be flexible and vary based on runtime conditions each time it is used.

For instance, one very simple use of an expression driven parameter in a JDBC Connection would be to define the User ID and Password as PROJECT Variables (e.g. `PROJECT.XPATH("USERCONFIG/MyDeployUser")`). This way when you deploy the project, you can update the PROJECT Variables in the Deployment Wizard to values appropriate for the final deployment environment. At the other extreme, you could have a custom script that queries a Java business object in the Application Server to determine what User ID and Password to use.

➤ To switch a parameter from Constant driven to Expression driven:

1. Click the right mouse button in the parameter field you are interested in changing.
2. Select **Expression** from the context menu and the editor button will appear or become enabled.
3. Click on the button and then create an expression that evaluates to a valid parameter value at runtime.



About JDBC Drivers and Connection Pools

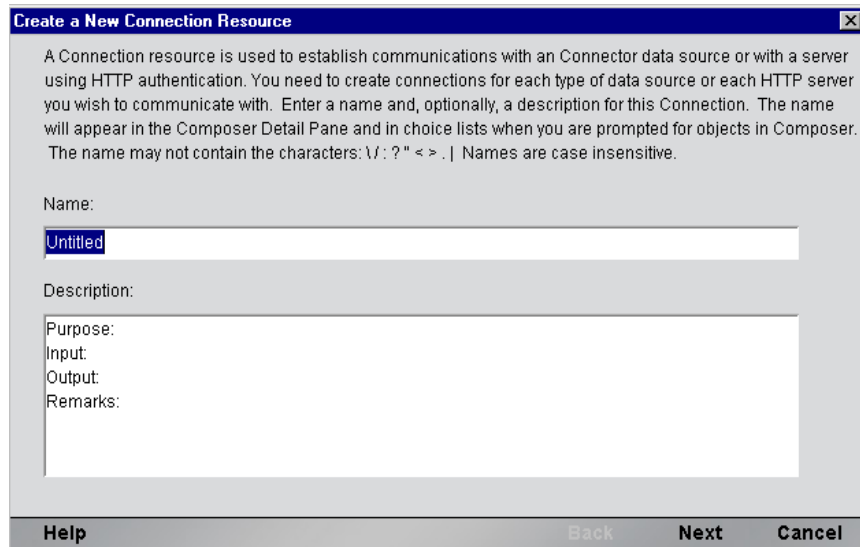
When you create a Connection Resource, you are asked to provide a Driver Name and Connection Pool.

The JDBC Driver **sun.jdbc.odbc.JdbcOdbcDriver** is part of the JRE (Java Runtime Environment, which you can find under the Composer directory), and you can use this driver to establish your connection. But you can also obtain other JDBC drivers. For instance, the SilverStream Application Server has its own JDBC drivers. Also, you can visit the Web site of the vendor for the SQL database you're using and download their driver(s).

A *connection pool* is a set of database connections managed by the application server for the various applications it manages. It provides more efficient use of database and connection resources for multiple applications running in the same application server. This, in turn, can improve overall system performance. You can obtain the Pool Name for your application server from your Server Administrator. For deployments within the SilverStream Application Server the pool name will be *Databases/DBName/DataSource* where *DBName* is the name that was used when the database was added to the server. For example, if you were connecting to the TutorialBegin3 database provided with the application server, the pool name would be *Databases/TutorialBegin3/DataSource*.

➤ **To create a JDBC connection resource:**

1. Select **File>New xObject>Resource>Connection**. The “Create a New Connection Resource” Wizard appears.



The screenshot shows a dialog box titled "Create a New Connection Resource". The dialog contains the following text: "A Connection resource is used to establish communications with an Connector data source or with a server using HTTP authentication. You need to create connections for each type of data source or each HTTP server you wish to communicate with. Enter a name and, optionally, a description for this Connection. The name will appear in the Composer Detail Pane and in choice lists when you are prompted for objects in Composer. The name may not contain the characters: \ / : ? " < > . | Names are case insensitive." Below this text, there is a "Name:" label followed by a text input field containing the word "Untitled". Below the name field, there is a "Description:" label followed by a larger text area. The text area is divided into four sections: "Purpose:", "Input:", "Output:", and "Remarks:". At the bottom of the dialog, there are four buttons: "Help", "Back", "Next", and "Cancel".

2. Type a **Name** for the connection object.
3. Optionally, type **Description** text.
4. Click **Next**.

Enter a Driver name for the database (e.g. com.sssw.jdbc.oracle8.Driver). Specify the driver specific URL for the database (e.g. jdbc:sssw:oracle:MYDB). Enter a connection pool provided by the application server after deployment. You may create more than one JDBC Connection. Checking 'Default' makes this Connection the initial selection when creating a JDBC Component. Use the Test button to check your connection. You may save connections that fail the test.

Connection Type: JDBC Connection

JDBC Driver

JDBC URL

User ID

Password

DB Params

Deployed Pool Name

Allow SQL Transactions ☒

Test

☐ Default

Help Back Finish Cancel

5. Select **JDBC Connection** from the **Connection Type** pull down menu.
6. In the **JDBC Driver** field, enter the name of the JDBC driver you want to use. For example, **com.sssw.jdbc.mss.odbc.AgOdbcDriver** for the SilverStream driver. (For more information see “About JDBC Drivers and Connection Pools” on page 14.)
NOTE This parameter, and all subsequent parameters in this dialog, can be dynamically set using Expressions. See “About Constant and Expression Driven Connection Parameters” earlier in this chapter.
7. In the **JDBC URI** field, enter the location of the database you want to reach. For example, **jdbc:sssw:odbc:XTutorial** where **jdbc:sssw:odbc:** is required syntax by the driver and **XTutorial** is an ODBC Data Source Name (DSN) defined on the specific computer where the component will run. (The DSN is defined externally from Composer by accessing the ODBC Administrator in the Windows Control Panel.) For deployment, you can maintain the connection described above, provided that the server allows for ODBC connectivity. The more likely scenario is that you will want to take advantage of the power of the application server in managing database access. In that case, you need to provide the connection pool name as described below.
NOTE The **JDBC Driver** and **JDBC URI** fields are both case sensitive.
8. Enter a valid **User ID** to sign on to the selected database.
9. Enter a valid **Password** for the selected database.
10. In the **DB Params** field, enter any database-specific parameters that might apply to your connection. For example, to allow updates to a SilverStream SQL Anywhere database,

enter **S3SqlAnywhereAuth=true** as a parameter. Note that parameters should be entered as **name=value** pairs. If more than one **name=value** param is specified, separate the pairs using semicolons, e.g., `param1=true;param2=true;param3=false`.

NOTE If no database-specific parameters will be used, enter `false` in this field.

11. Enter a **Pool Name** if required. For more information, see “About JDBC Drivers and Connection Pools” on page 14.

NOTE Connection pooling is only operational in the deployment environment. Setting the name here will not affect Composer connections. Only the deployed project will be affected.

12. Check the **Allow SQL Transactions** checkbox if you intend to exercise direct control over transactions (using SQL Begin, Commit, and Rollback verbs) in your component’s Action Model.

Checking the **Allow SQL Transactions** box has a number of effects:

—It turns auto-commit *off* for the JDBC driver. (The state of the auto-commit flag is restored, however, at the end of the transaction, before returning the connection back to the pool.)

—It causes all SQL commit and rollback commands to be translated to the corresponding JDBC connection calls.

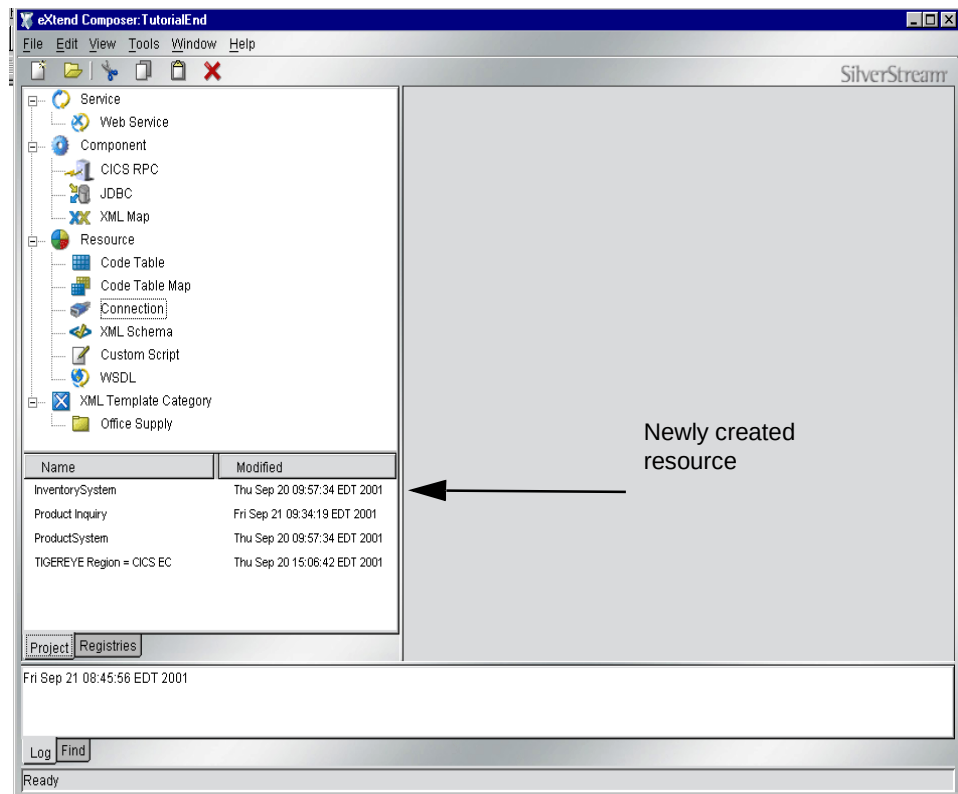
—It causes Composer Enterprise Server to check the final Execute SQL Action in the component to see that the final action is a commit or a rollback. If the final action is *not* a commit or rollback, Composer Enterprise Server performs a rollback by default, so that a dirty connection (that is, a connection with uncommitted changes) is not inadvertently returned to the pool.

NOTE For a further discussion of the **Allow SQL Transactions** checkbox, see the Transactions chapter of the *eXtend Composer Application Server Guide* for your application server.

13. Check the **Default** checkbox if you would like to use the current connection as the default connection for any new JDBC Components you create in your project.
14. Click **Test** to see if your connection is successful. A “success” or “failure” message appears for your connection. *You can continue creating the resource, even if your connection fails.*

NOTE This does not test the connection pool (if defined).

15. Click **Finish**. The newly-created resource connection object appears in the Composer Connection Resource detail pane.



Creating XML Templates for Your Component

In addition to a connection resource, a JDBC component also requires that you have already created XML templates so that you have sample documents for designing your component. (See Chapter 5, *Creating XML Templates*, in the *Composer User's Guide* for more information.)

Also, if your component design calls for any other xObject resources such as custom scripts or code table maps, it is best to create these before creating the JDBC Component. For more information, see *Creating Custom Scripts* in the *Composer User's Guide*.

3 Creating a JDBC Component

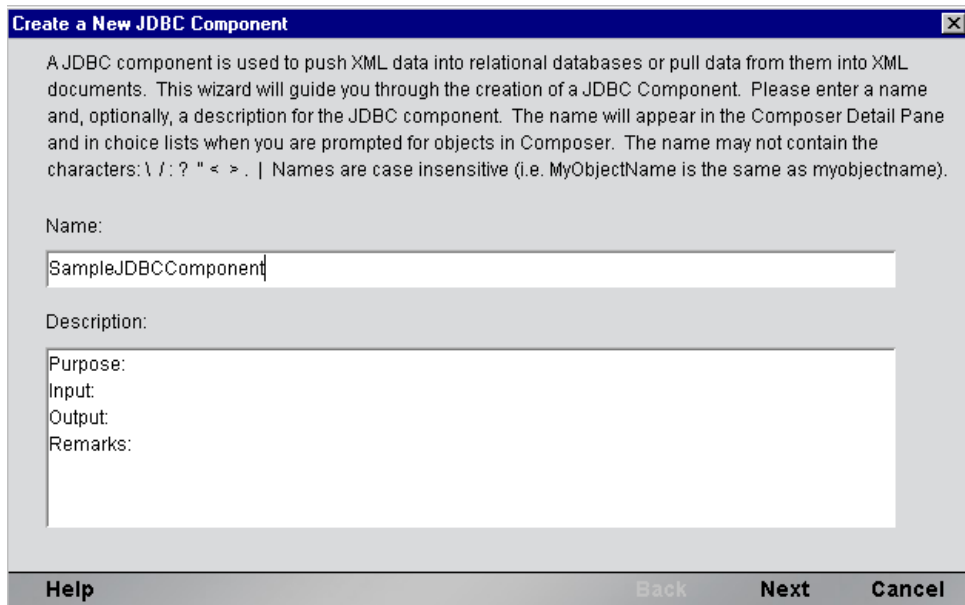
Before Creating a JDBC Component

As with all eXtend components, the first step in creating a JDBC component is to specify the XML templates needed. (For more information, see *Creating a New XML Template* in the *Composer User's Guide*.) Once you've specified the XML templates, you can create a component, using the template's sample documents to represent the inputs and outputs processed by your component.

Also, as part of the process of creating a JDBC component, you can select a JDBC connection or you can create a new one. If you create the connection beforehand, then it is available to all new JDBC components as a selection. (See "Creating a JDBC Connection Resource" on page 13.)

➤ **To create a new JDBC component:**

1. Select **File>New xObject>Component>JDBC**. The "Create a New JDBC Component" Wizard appears.



The screenshot shows a dialog box titled "Create a New JDBC Component" with a close button (X) in the top right corner. The dialog contains the following text: "A JDBC component is used to push XML data into relational databases or pull data from them into XML documents. This wizard will guide you through the creation of a JDBC Component. Please enter a name and, optionally, a description for the JDBC component. The name will appear in the Composer Detail Pane and in choice lists when you are prompted for objects in Composer. The name may not contain the characters: \ / : ? " < > . | Names are case insensitive (i.e. MyObjectName is the same as myobjectname)." Below this text are two input fields. The first is labeled "Name:" and contains the text "SampleJDBCComponent". The second is labeled "Description:" and contains a list of labels: "Purpose:", "Input:", "Output:", and "Remarks:". At the bottom of the dialog are four buttons: "Help", "Back", "Next", and "Cancel".

2. Enter a **Name** for the new JDBC Component.

3. Optionally, type **Description** text.
4. Click **Next**. The XML Property Info panel of the New JDBC Component Wizard appears.

Create a New JDBC Component

Specify one or more XML Templates to help design Input to this Component or Web Service and only one to design Output. The sample XML Documents in each Template are design time aids to help you build Action Models for the component. The samples are not actually used at runtime after deployment to your application server. The Identifier is fixed and represents the name used to refer to the XML Document during component execution. Selecting System {ANY} allows you to use an empty template (i.e. accept any document as Input).

Identifier	Template Category	Template Name
Input	Office Supply	ProductRequest

Buttons: Add, Delete

Output: Office Supply, ProductResponse

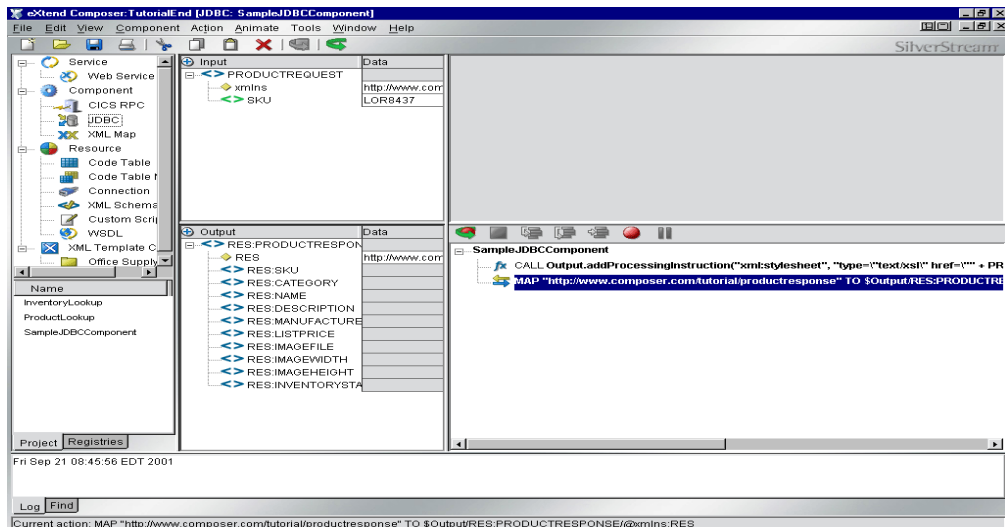
Buttons: Help, Back, Next, Cancel

5. Specify one or more Input templates as follows:
 - Select a **Template Category** if yours will be different than the default category.
 - Select a **Template Name** from the list of XML templates in the selected **Template Category**.
 - To add additional input XML templates, click **Add** and repeat steps 2 through 4.
 - To remove an input XML template, select an entry and click **Delete**.
6. Select an XML template for output. (Note that there is only *one* output DOM, whose name is Output.)

NOTE You can specify an output XML template that contains no structure by selecting **{System}{ANY}** as the Output template. For instance, you might do this if you want to generate a customized Output DOM using the “Custom” option of the Result Mapping tab. (For more information, see “Creating an Output DOM without Using a Template” in the *Composer User’s Guide*, in the chapter on creating an XML Map Component.)

- Click **Next**. The Connection Info panel of the “Create a New JDBC Component” Wizard appears.

- Select a **Connection** type from the pull down list. For more information on the JDBC Connection, see “Creating a JDBC Connection Resource” on page 13.
- Click **Finish**. The component is created and the JDBC Component Editor appears.

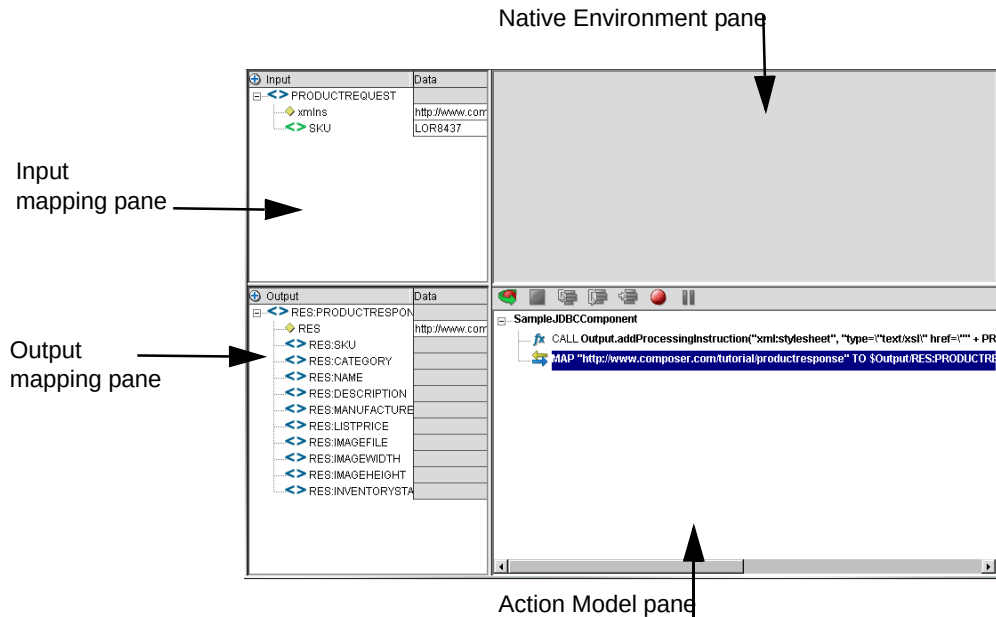


About the JDBC Component Editor Window

The JDBC Component Editor includes all the functionality of the XML Map Component Editor. It contains mapping panes for Input and Output XML documents as well as an Action pane.

The difference, however, is that the JDBC Component Editor also includes a Native Environment pane common to all Connects. It appears as a grey pane until you create an SQL Statement action, at which time it is populated with the Query pane (see p. 22), which is specific to the JDBC connector.

NOTE To display the Query Pane, you must first select **SQL Statement** from the **Action** menu and create an SQL action. Otherwise, the pane remains greyed out.



About the Query Pane

When the Query pane (i.e., the activated Native Environment pane) is displayed, it becomes a fully functional SQL environment for creating and testing queries in real time. From this pane, you can perform the following:

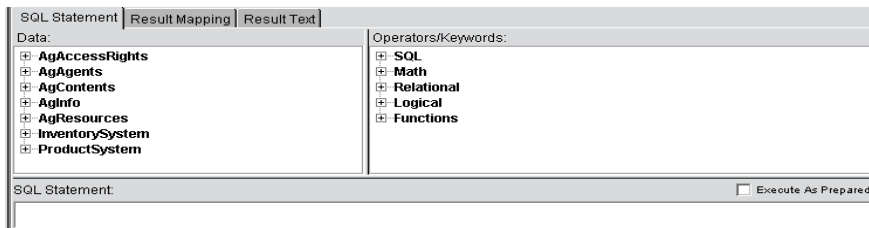
- Take data from an Input XML document (or other available DOM) and use it to create or modify an SQL Query against a relational data source

- Take the results of that query and put it into a DOM (e.g., Temp, Output, MyDom, etc.)

The Query pane includes three tabs: the SQL Statement tab, the Results Mapping tab, and the Results Text tab.

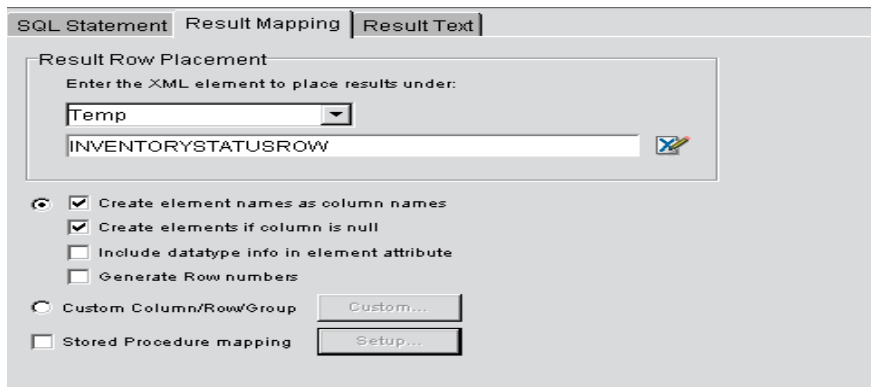
SQL Statement Tab

When the Query pane first opens, it displays the SQL Statement tab in a live SQL environment. The SQL Statement tab allows you to write or build SQL commands.



Result Mapping Tab

The **Result Mapping** tab allows you to map the result of your database query into an XML document. It also allows you to designate the exact XML branch element under which you'd like the query result to appear. The Result Mapping tab is shown below.



Result Text Tab

The **Result Text** tab displays the actual SQL statement sent and the data that was returned following the execution of the database query. This is helpful if errant data shows up in a temporary or output DOM. You can compare the data from the Result Text tab with the data in the DOM to see where the error occurred. The Result Text tab is shown below.



4 Performing JDBC Actions

About Actions

An *action* is similar to a programming statement in that it takes input in the form of parameters and performs specific tasks. Please see the chapters in the *Composer User's Guide* devoted to Actions.

Within the JDBC Component Editor, a set of instructions for processing XML documents or communicating with non-XML data sources is created as part of an *Action Model*. The Action Model performs all data mapping, data transformation, data transfer between SQL databases and XML documents, and data transfer within components and services.

An Action Model is made up of a list of actions. All actions within an Action Model work together. As an example, one Action Model might contain individual actions that read invoice data from a disk, retrieve data from an inventory database, map the result to a temporary XML document, make a conversion, and map the converted data to an output XML document.

The Action Model mentioned above would be composed of several discrete actions. These actions would:

- Open an invoice document and perform an SQL command to retrieve invoice data from a database
- Map the result to a temporary XML document
- Convert a numeric code using a Code Table and map the result to an Output XML document

The SQL Statement Action

The SQL Statement action is most commonly used to query an existing database and then map the result to an XML document. However, the full set of SQL Data Manipulation Language (DML) statements can be utilized (including database inserts, deletes, and updates).

NOTE You should be familiar with SQL database commands in order to form a valid query with the SQL Statement action.

Prepared Statements

The JDBC Connect has the ability to prepare (or precompile) SQL commands and cache them in memory so that when the same command executes over and over again (for example, in a

loop), the cached statement can be reused, with new argument values inserted as need be. This can be a significant performance optimization in cases where statements execute many times.

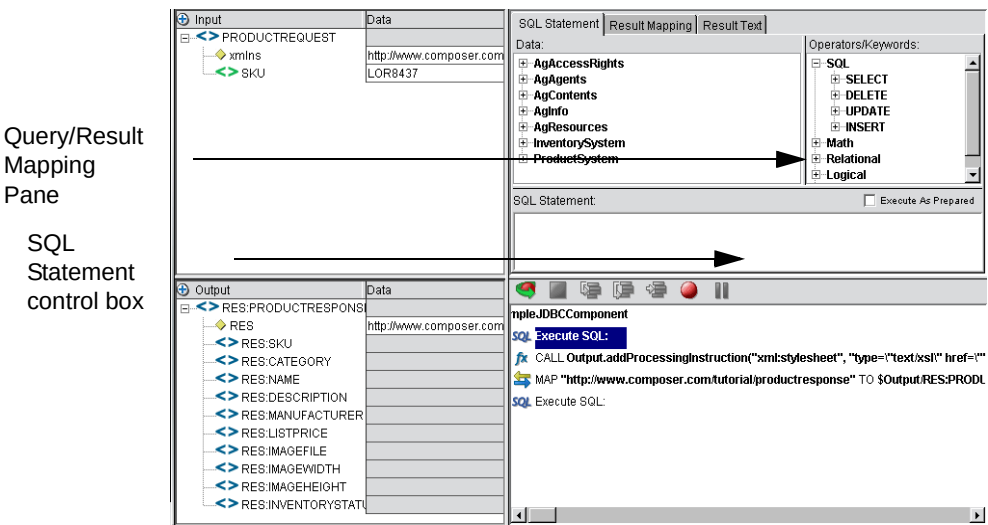
You can designate any SQL statement as a “prepared statement” using the checkbox provided in the Query/Result Mapping Pane, just above the SQL edit box:



By default, this checkbox is unchecked. For SQL Statement actions that are executed only once in the course of a service’s lifetime, it is recommended that you leave the checkbox disabled. For statements inside loops, the checkbox can be checked.

NOTE You may want to do some benchmarking to determine whether and to what degree using the **Execute as Prepared** checkbox is beneficial in a given application.

The creation of SQL statements for use in JDBC Components is done inside the Query/Result Mapping Pane.



If you are editing a previously created action model that already contains SQL Statement actions, you can make the Query/Result Mapping Pane come into view simply by selecting (clicking on) any existing SQL Statement action. Otherwise, you will create an SQL Statement action.

➤ **To create an SQL Statement action:**

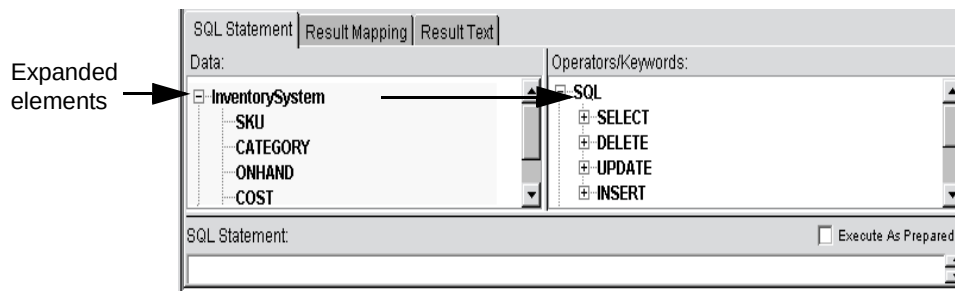
1. Create or open a JDBC Component.
2. Highlight a line in the Action Model where you want to place the SQL Statement action. The new action will be inserted below the line you highlight.
3. From the **Action** menu, select **New Action**, then **SQL Statement**. The Query/Result Mapping pane appears in the Native Environment pane of the JDBC Component Editor window, as shown above.

Building an SQL Statement

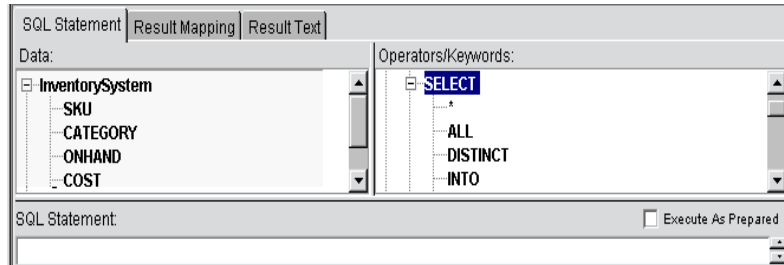
Building an SQL Statement involves bringing together data, operators, and keywords.

➤ **To build an SQL Statement:**

1. Place the cursor in the **SQL Statement** control box in the Query/Result Mapping pane.
2. Expand the **Data** columns and/or the **Operator/Keywords** by clicking the plus signs.



3. Double-click each **Data** column and/or **Operator/Keyword** that you would like to add to the **SQL Statement** box. When you double-click an item, it automatically appears in the **SQL Statement** box at the insertion point.



4. Optionally, you may drag elements from an open DOM tree (e.g., the Input DOM pane) into the **SQL Statement** box.
5. Optionally check the **Execute as Prepared** checkbox. (See discussion further above, under “Prepared Statements”.)

Building an Example Query

Here is an example SQL statement:

```
SELECT * FROM ProductSystem WHERE SKU =  
' :Input.XPath ( "PRODUCTREQUEST/SKU" ) ' ;
```

In order to build this statement, the component must satisfy the following:

- The component must be able to use a (previously defined) connection resource to connect to the database
- The database must have a table called ProductSystem that has a column called SKU
- The component must have a template containing a sample XML document with a root element, PRODUCTREQUEST, that has a child element named SKU

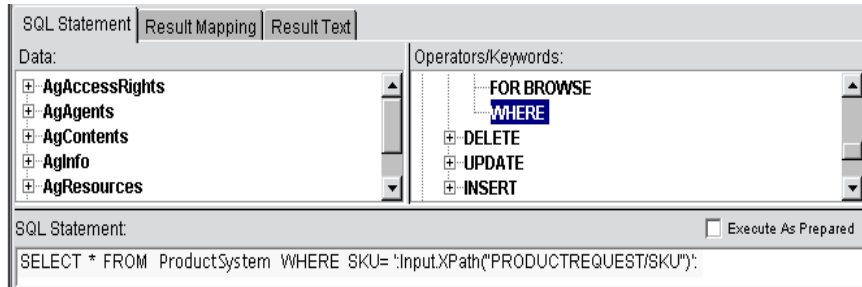
This example statement, in plain English, means:

“Select all columns from the database’s ProductSystem table where a record’s value in column SKU is equal to the content of the Input DOM’s PRODUCTREQUEST/SKU element.”

➤ To build the example statement:

1. Expand the **SQL** tree in the Expression builder and double-click **SELECT**.
2. Double-click ***** in the Expression Builder.
3. Double-click **FROM** in the Expression Builder.
4. Type **ProductSystem**.
5. Double-click **WHERE** in the Expression Builder.

6. Type **SKU =**.
7. Select **SKU** in the Input DOM and drag it into the SQL Statement control.
8. Optionally type a semicolon (;) at the end of the SQL Statement.
9. Select **File>Save**. The Query/Result Pane should look like this:



Using Stored Procedures

Many RDBMS vendors provide the ability to execute procedural code stored in the RDBMS system. Using these stored procedures allows for high-performance interfaces that are independent of the underlying table implementations.

Using stored procedures can be helpful in controlling access to data. User access to data can be limited to the scope of the stored procedure. Limiting access to data with stored procedures preserves data integrity by insuring data is entered in a consistent manner. Stored procedures also improve efficiency. They're memory resident, which speeds execution. Their use decreases network traffic. Productivity is improved via their use since stored procedures only need to be written and debugged once but can be reused by many.

While often used interchangeably, for the sake of discussion we'll differentiate between the terms Procedures and Functions. A Procedure is a subroutine that doesn't necessarily return any data but may via the call's parameters or as external result sets. A Function, on the other hand, always returns *something*. Both Procedures and Functions can pass Parameters.

eXtend Composer allows you to map parameters to stored procedures and functions, execute stored procedures and functions and map returned data to DOM/node combinations.

Syntax Requirements

In order to package the Procedure or Function call correctly, eXtend Composer requires certain formatting conventions be followed. For example:

{ – indicates that a call to a Function or Procedure follows

} – indicates the end of a call to a Function or Procedure

The syntax for procedures and functions support parameters which may be Expressions, Placeholders or Constants.

Expression: Expressions may be used pass variable input data to a procedure or function. Expressions used as parameters in procedure and function calls are preceeded with a colon (:) and enclosed in single quotes. (e.g. ‘:<variablename>’).

Question Mark: Question Marks (?) may be used as parameters and serve as placeholders to which the procedure returns data. A question mark is also used for the result in a function.

Constant: Constants are used to pass input data in procedures and functions but, unlike expressions or placeholders cannot be used to accept returned data. Literal values are enclosed in single quotes.

Rules for Stored Procedure Parameters

Stored procedures may have Input Parameters, Input/Output parameters and Output Parameters.

Input Parameters: Input Parameters pass data to stored procedures. Input Parameters may be Constants or Expressions.

Input/Output Parameters: Input/Output Parameters pass data to stored procedures and accept data returned from stored procedures. Input/Output parameters must be Expressions.

Output Parameters: Output Parameters accept data returned from stored procedures. Output parameters may be either an Expression or a Question Mark as a placeholder.

Using Procedures and Functions in a JDBC Component

For all the examples below the following steps should be executed.

- Add a new SQL action
- **Execute as Prepared** is set to true (check the checkbox; see “Prepared Statements” on page 19).

NOTE For mapping the results of stored procedures, see Chapter 6.

Syntax for running a Procedure from within eXtend Composer

Procedures that do not return a value:

```
{ call [<packagename>.]<procedurename>([param1,  
param2...,paramn]) }
```

Example:

```
{ call composerDemoPackage.sp1_withParams('12345','George') }
```

Procedures that return a result set:

```
{ call [<packagename>.]<procedurename>([param1,  
param2...?...paramn]) }
```

where ? is a parameter to which the result set is returned. A result set may also be returned to other parameters which contain Expressions.

Example:

```
{ call composerDemoPackage.sp_withParams('93324', ':FirstName',  
?) }
```

In this example '93324' is a constant, ':FirstName' is an Expression and ? is a placeholder.

NOTE Only Oracle returns result sets as parameters. Non-Oracle RDBMSs may return result sets but, not as parameters.

Backward Compatibility for Oracle Procedures that return a result set:

Prior to version 4.0, eXtend Composer provided support for Oracle Procedures that return result sets as parameters. To do so, eXtend Composer (prior to version 4.0) required the user to specify the Oracle Cursor Position within the procedure call. The pre-Composer 4.0 syntax included **ocp:n** – where **ocp** stands for Oracle Cursor Position and **:n** indicates which parameter contains

the cursor. This syntax was used in pre-4.0 versions of eXtend Composer and is maintained in version 4.0 and greater for backward compatability.

```
{ call [<packagename>].<procedurename>([param1,  
param2...ocp:x...paramn]) }
```

Example:

```
{ call composerDemoPackage.sp_withParams('93324', 'Melissa',  
ocp:3) }
```

NOTE The contents of the result set will be returned in the same manner as a standard SELECT statement. The results will be automatically be mapped to the selected XML Document. The defaults are Output as the Document and RESULTINFO/ROW as the XPath location.

Syntax for Calling a Function from within Composer

Functions that return a result set:

```
{ ? = [<packagename>].<functionName>([param1, param2...,paramn]) }
```

Example:

```
{ ? = call composerDemoPackage.fn_justOneReturn( ) }
```

Backward Compatability for Oracle Functions that return a result set:

To provide backward compatability with pre-4.0 versions of eXtend Composer, the following syntax will continue to be supported in eXtend Composer 4.0 and greater.

```
{ ocp:1 = [<packagename>].<functionName>([param1,  
param2...,paramn]) }
```

Example:

```
{ ocp:1 = call composerDemoPackage.fn_justOneReturn( ) }
```

Other Methods of Calling Functions for Specific Tasks

You may call any function that does not update the database from within a select statement.

Example:

```
select fn_addMin(4,6) "Sum" from dual
```

To use a function that does not return a result set but updates the database, call it from within a function that *does* return a result set – see the example `fn_callAddMin`

Example:

```
{ ? = call composerDemoPackage.fn_callAddMin(22,44 ) }
```

Colons in SQL Statements

Colons are special characters in SQL Statements, because eXtend Composer treats colons as markers indicating the presence of ECMAScript immediately to the right. In the above action, the SQL Statement includes the string

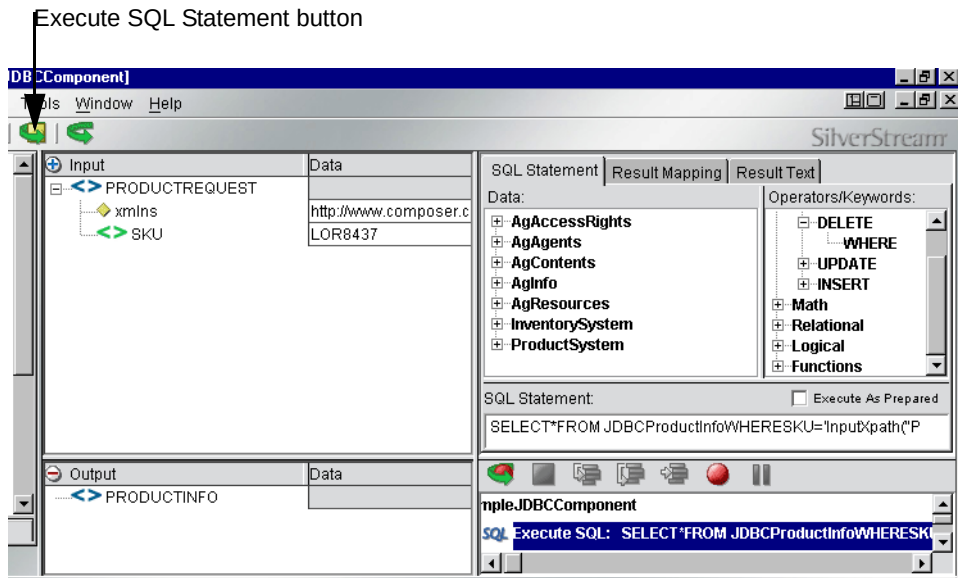
```
` : Input.XPath ( "PRODUCTREQUEST/SKU" ) `
```

which contains a colon followed by an ECMAScript expression involving the `XPath()` method. Without the colon, the string would be evaluated as a string-literal. With the colon, it is evaluated as an ECMAScript expression.

NOTE If you need to use colons as literal values inside SQL Statements, escape every occurrence of a literal colon with a backslash. Otherwise, you may see errors.

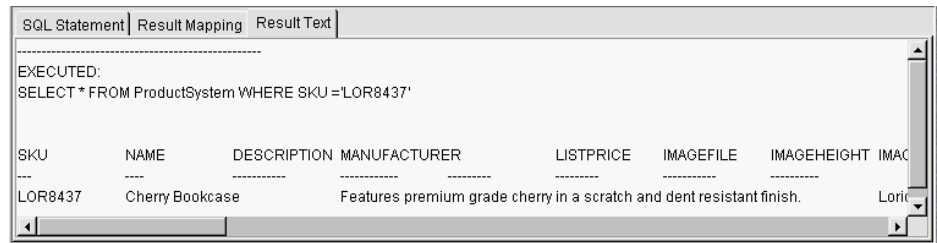
Executing the SQL Statement

After you have built the SQL Statement, click the **Execute** button.



Checking the Results

You can check the results of your SQL statement by looking at the data retrieved in the familiar row and column format. To do so, click the **Result Text** tab.



If the query result returned by the SQL statement looks correct, you can continue designing your component's Action Model. Otherwise, you can return to the SQL Statement tab and debug your SQL as necessary.

Mapping Results into the Output DOM

Use the **Result Mapping** pane to select where to place the rows and columns of your results into the DOM tree.

➤ To use Result Mapping:

1. Select the **Result Mapping** tab in the **Query/Results Mapping** pane. The Results Mapping pane appears.

2. Under **Result Row Placement**, select the destination DOM to which you would like the result of the SQL query mapped.
3. Next, select the DOM element under which you'd like each result row to appear. If an appropriate DOM is not listed, you may add another XML template using the **File>Properties>XML Properties Info** dialog from the menu. Or you can add a Temp DOM by choosing **Components>Manage Temp Documents**.
4. Select options as follows:

Default Result Mapping: Choose the first radio button for standard Column/Row/Group mapping:

 - **Create element name as column name.**
 - **Create elements if column is null.** This creates XML elements with empty content if the column returned has no data.

- **Include data type info in element attribute.** This creates an attribute for each element indicating the data type of the result column.
- **Generate row numbers** (if applicable).

Custom Result Mapping: Choose the second radio button, Custom Column/Row/Group, to perform custom column, row, or group mapping (see Chapter 5).

Stored Procedure Mapping: Choose Stored Procedure mapping to map data returned from stored procedures. (see Chapter 6).

5. Select **File>Save**.

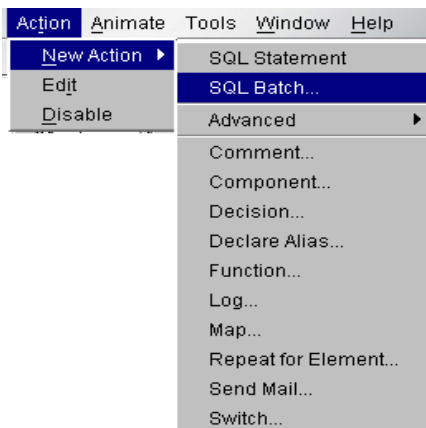
The SQL Batch Action

Most database drivers allow batch execution of SQL statements in order to minimize demand on connection resources. For example, a user may want to insert data into a table in one database and delete data from a table in another database, all in one round trip. This is possible with the SQL Batch action.

SQL Batch actions allow you to specify that a particular group of SQL Statement actions should be accumulated into a single batch and transmitted to the database as a unit.

NOTE SELECT operations may not be used in batches. Use only INSERT, DELETE, and UPDATE statements.

To access the SQL Batch action, right-click inside the action pane and choose **New Action > SQL Batch** as shown below.



There are three SQL Batch commands, each of which places a new action in the action model: Start Batch, Execute Batch, and Discard Batch.

Start Batch

You must tell Composer where the beginning of a batch occurs, by placing a Start Batch statement before the first SQL Statement in a series of statements that you want to group. This command sets a checkpoint for rollback purposes (in case the batch does not finish normally).

From the first occurrence of this command until the next occurrence of an Execute Batch command (see below), SQL Statements are merely accumulated, rather than executed.

Execution of a batch does not occur until an Execute Batch command is reached.

Regular (non-SQL) actions, such as Map and Function actions, are not affected by Batch operations. If you place Map actions, Function actions, or any other non-SQL actions within or after a group of batched SQL Statement actions, those actions will execute *before* the SQL Statements in your batch, because the batch cannot execute until an Execute Batch is reached.

Execute Batch

An Execute Batch command causes all SQL Statements in a batch to be sent, as a unit, to the database. (If no Execute Batch command is issued, none of the SQL Statements in the preceding batch will get executed.)

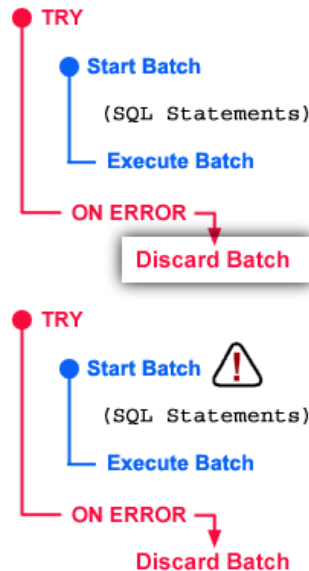
An Execute Batch statement can be placed immediately after a batch of SQL Statement actions, or it can be placed at some point downstream of the batched actions (possibly in one branch of a Decision action). In other words, you can create a batch in one location and execute it, conditionally, from another location in your action model.

Discard Batch

The Discard Batch command is a memory-deallocating command that causes the previously held batch to go out of scope. It frees the memory held by the preceding batch.

Ordinarily, when an SQL batch executes without error, the batch is discarded automatically after it executes and there is no need to issue an explicit discard. You would use Discard Batch when you have an action model that contains two or more sequential SQL batches (each with its own Execute Batch command) wrapped in Try/On Error statements. The need for the Discard Batch arises when one of the upstream batches executes abnormally (generating an exception). In order to continue to another batch, you need to purge the previous batch from memory (with a Discard Batch in the On Error branch of the “Try” action). Failure to use Discard Batch under

these conditions would cause the next Start Batch to throw an exception. This scenario is shown in the illustration below.



In the case depicted above, where there are two SQL batches (each enclosed in a Try/On Error action), failure to include a Discard Batch action in the error branch of the first Try will cause the next Start Batch to throw an exception (assuming the first batch fails).

In summary: When two or more batches will execute sequentially, wrap each in a Try/On Error action and *include a Discard Batch command in the On Error branch of each*.

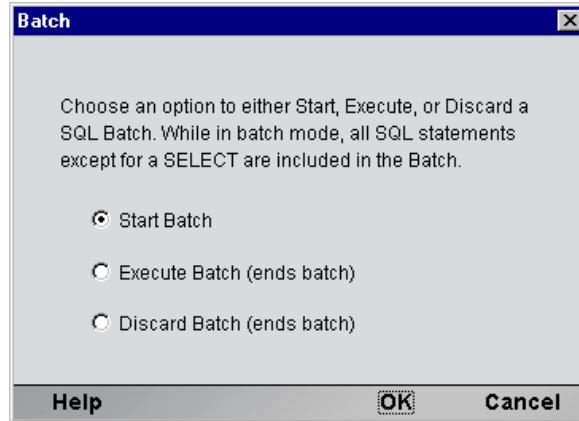
For action models in which there is only a *single* SQL batch, Discard Batch is not necessary. After normal execution of a (single) batch, memory allocated to the batch is released automatically; and if the batch returns an error, the batch will go out of scope (and be garbage-collected) when the component itself goes out of scope.

Creating Batch actions

Batch actions are created using the SQL Batch menu command (available from **Action > New Action > SQL Batch** in the JDBC Component Editor main menu, or via **New Action > SQL Batch** in the contextual menu).

➤ **To create a SQL Batch action:**

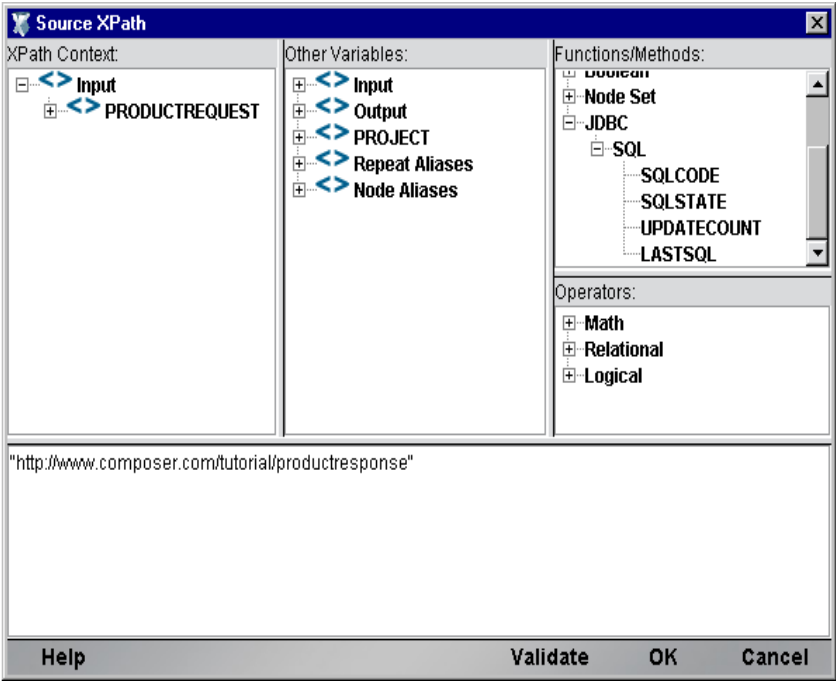
1. Place the cursor in a line preceding the group of SQL Statements that you want to batch. Then press the right mouse button and select **New Action > SQL Batch**. The Batch setup dialog appears.



2. Choose the **Start Batch** radio button to insert a Start Batch command in your action model. Otherwise, choose **Execute Batch** or **Discard Batch**, as appropriate.
3. Click **OK** to dismiss the dialog. A new action appears in your action model.

JDBC-Specific Expression Builder Properties

SQL queries can result in certain status and/or error values being returned (for example, the number of records that were changed by an Update). Often, it is useful to be able to reference these values in ECMAScript expressions. The Expression Builder pick list (in the top portion of the Expression Editor window) contains properties that are specific to JDBC Actions involving SQL: namely, `SQLSTATE`, `SQLCODE`, and `UPDATECOUNT`. (See panel below.)



Using Other Actions in the JDBC Component Editor

In addition to the SQL Statement action, you can use all other actions in the JDBC Component Editor. The **Action** menu lists both basic and advanced actions, as described in the following tables.

Table 4-1

Basic Actions	Description
Comment	Documents the Action Model. You can use comments to clarify the processing, especially if Decisions and/or Repeats are used in the Action Model.
Component	Executes another component and defines runtime DOMs to be passed to, and received from the called component.

Basic Actions	Description
Decision	Allows you to execute one of two sets of actions based on a condition you specify. Processing branches along a True or False path, depending on how your condition is resolved as the component executes.
Function	Executes either an ECMAScript function or a custom script you have previously created. You can create custom scripts using Composer's Custom Script Resource Editor. (Refer to Chapter 9 of the <i>Composer User's Guide</i> .)
Log	Writes information to various log files specified in the component. There are three Log types: System Output, System Log, and User Log.
Map	Transfers and optionally transforms element data from one XML DOM to another.
Send Mail	Automatically sends an e-mail to a specified e-mail address during execution of the component.
Switch	Allows program control to branch to a particular block of actions based on a match between an input value and a Case value. This is essentially a convenience action that can be used to eliminate long, hard-to-read if/else (Decision action) chains.

The actions in Table 4–2 are located on the component editor's **Action** menu, under the **Advanced**, **Data Exchange** and **Repeat** sub-menus.

Table 4-2

Advanced Action	Description
Apply Namespaces	Provides a way to override NameSpace prefixes, declare a new one or ignore a NameSpace altogether.
Raise Error	Evaluates a condition which if true, writes the contents of an expression to a global variable called ERROR. If used alone, it throws an exception, stops a component, and returns control to the service. If used within the Execute branch of a Try On Error action, it is evaluated and control passes to actions in the On Error branch.
Simultaneous Components	Allows two or more components to be executed simultaneously (that is, in multithreaded fashion).
Transaction	Allows you to invoke <i>User Transaction</i> commands (such as <i>begin</i> , <i>commit</i> , and <i>rollback</i>) in components that will be deployed as part of non-Container-managed services, or <i>setRollbackOnly</i> in components that will be part of Contained-managed EJB deployments.
Try On Error	Responds to actions that produce errors by executing a set of actions. The Try On Error action is essentially an error trapping and solution action.
XSLT Transform	Transforms an XML file according to instructions in an XSL file. The output is commonly used for rendering XML files in the Web browsers.

Table 4-3

Data Exchange Actions	Description
UR_/File Read	Allows a file format that is not XML to be read into Composer
UR_/File Write	Allows a file to be written into another format from XML.

Data Exchange Actions	Description
WS Interchange	Executes a Web Service using messages and operations defined in a WSDL resource.
XML Interchange	Reads external XML documents into the component's DOM or writes the component's DOM to an external XML document. Read/write methods include: Get, Put, Post, and Post with Response using the File, FTP, HTTP, and HTTPS protocols.

Table 4-4

Repeat Actions	Description
Break	Stops execution of a Repeat for Element, Repeat for Group, or Repeat While loop and continues execution with the next action outside the loop.
Continue	Stops execution of the current Loop iteration in a Repeat for Element, Repeat for Group, or Repeat While loop, and continues at the top of the same loop with the next iteration.
Declare Group	Allows you to create and name a group based on an element that occurs multiple times. Groups are used in the Repeat for Group action.
Repeat for Element	Repeats one or more actions for each occurrence of a specified element in your DOM tree. The Repeat For Element action allows you to create a loop based on an element that occurs multiple times.
Repeat for Group	Repeats one or more actions for each member of a group. A Repeat For Group action allows you to re-structure your data and calculate aggregates on your data.
Repeat While	Repeats one or more actions by creating a loop. A While Repeat action allows you to base a processing loop on any valid ECMAScript expression.

Handling Errors and SQL Messages

SQL returns certain coded values when errors occur (i.e., no record was found in a Query) or as a report on the result of certain actions (i.e., how many records were changed by an Update). These results appear on the Result Text tab as three special variables labeled:

- SQLSTATE
- SQLCODE
- UPDATECOUNT
- LASTSQL

These variables are available to ECMAScript functions you may write and can be used for error handling within your JDBC component. For instance, you can create a Decision action to process after an SQL statement. Based on the value returned in the UPDATECOUNT variable, you can choose one or the other set of actions in the two branches of the Decision action. Likewise, error information contained in SQLSTATE or SQLCODE (which are standard SQL status variables) can be used to branch to appropriate recovery logic in case of error.

The LASTSQL variable is an eXtend-defined string variable which contains *the last SQL statement to actually execute* in the component in question. Logging the value of this variable can be useful for troubleshooting.

5 Using Custom Result Mapping

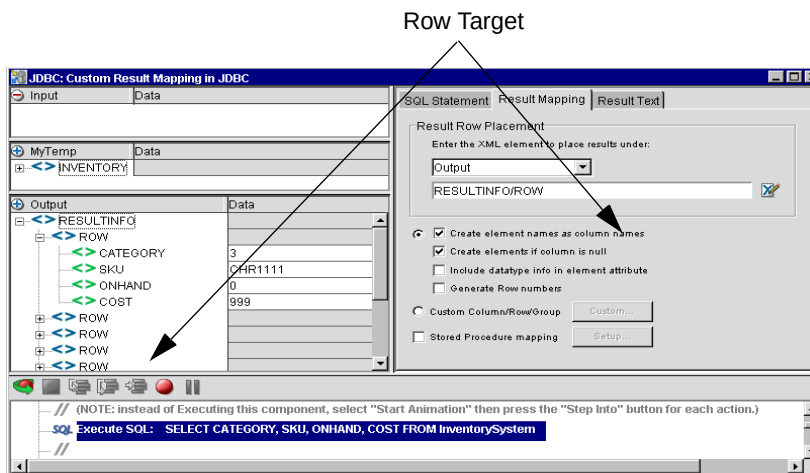
The following sections describe the similarities and differences between default and custom result mapping for the Execute SQL action. Custom mapping features are described in detail.

About Default Result Mapping

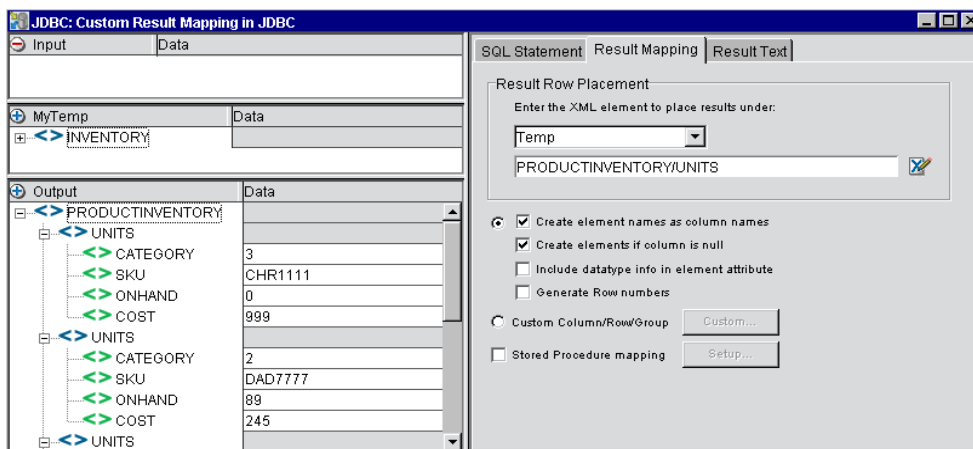
NOTE The examples cited in this section can be found in the “ActionExamples” project included in the Samples directory of your eXtend Composer installation.

The mapping of data returned from an Execute SQL action is determined by specifications on the Result Mapping tab in the SQL Mapping pane. The two **Result Row Placement** controls allow you to determine where in the target document to place the result set data. The drop down list specifies the DOM or Repeat alias context and the Expression edit box specifies the XPath location within the Context.

The Context is either the name of a DOM in the component or the name of a Repeat alias already specified in the component (where the Repeat alias itself represents a DOM context and XPath location). The Expression edit box specifies an XPath, the last element of which acts as the parent element for the returned results and will receive the data. The last element that receives the data is called the **Row Target**. If multiple rows are returned, then multiple Row Targets will be created. Each column returned in a row will appear as a child element of each Row Target.



By default, the Row Target is named “ROW” and is a child of a root element named “RESULTINFO” but you can change these names if you like. For instance, you may choose the Row Target to be PRODUCTINVENTORY/UNITS as shown in the graphic below.



Characteristics of the Default Result Mapping specifications are:

- Target element names created in the document are the same as column names returned in the result set
- All columns returned in the result set are mapped to the target document
- All columns are mapped to the same parent target element
- All rows are placed into a single document

NOTE Any column names that contain spaces will have the spaces replaced with the underscore character since XML does not permit spaces in element names.

About Custom Result Mapping

Use custom result mapping to:

- Create target element names different than the column names returned
- Map columns to different row targets
- Group the result set data by one or more columns
- Map only group information
- Map group and detail information

Custom result mapping is accessed via the **Custom...** button on the Result Mapping tab. Selecting this button displays a dialog with three tabs labeled Map Target, Detail Rows, and Declare Group/Repeat.

About Custom Result Mapping and Aliases

eXtend's default mapping behavior is to iterate through a list of one or more nodes (i.e., elements specified by an XPath pattern) from a source document, and map them to a single target document XPath location. If the target location doesn't exist, eXtend creates it. If you know the source list is greater than one, you must indicate to eXtend Composer whether you wish to map to the same physical target location for each member of the source list (i.e., overwrite the data in the specified physical target location), or create a new physical target location for each member of the source list (i.e., add new target locations as the repeated source is mapped). You indicate that you want to map each member of the source list to the same physical target location by specifying the Context as an actual DOM name. You indicate that you want to map each member of the source list to a *new* physical target location by specifying the Context using an alias.

NOTE This is also true for the Repeat for Element and the Repeat for Group actions.

You can think of the multiple rows of data returned by a SELECT statement as a repeating set of elements in an XML document. In that case, you may choose to create a Declare Group action creating a list of Groups and Detail elements within the Groups. Then you would create a Repeat for Group action to process the Group list or detail of each Group. The Custom Map Target, Detail Rows, and Declare Group/Repeat tabs provide a similar alias ability for repeating rows in SQL result sets as the Declare Group and Repeat for Group actions do for repeating elements in a document.

Using the MapTarget Tab

The **Map Target** tab is used to:

- Create your own target element names for each result set column
- Specify a target Context for each result set column

The Map Target tab controls the mapping of *each* returned row's individual columns. For each column, you specify a Context – Target XPath combination. The Context – Target XPath combination is specified for each column in the order they are listed in the projection list for the SELECT statement in your Execute SQL action. You cannot use Custom Result Mapping without filling in the Map Target tab.

Context: this specifies the target document for the column. The Target XPath will be appended to the Context to produce the full XPath location for the column in the target document. The Context can be a:

- *Document* – You may use this choice if your result set contains only one row, otherwise each additional row will overwrite the previous row's data.
- *Detail Alias* – A Detail Alias is defined on the Detail Rows tab and consists of a Document name and partial Target XPath. Or the Detail Alias may consist of a Group Alias (defined on the Declare Group/Repeat tab) and partial Target XPath location. Using a Detail Alias tells eXtend Composer to create a new physical target location for each member of the source list (i.e., each row in a result set).
- *Group Alias* – A Group Alias is defined on the Declare Group/Repeat tab and consists of a Document name and partial XPath location. Using a Group Alias tells eXtend Composer to create a new physical target location once for each Group in the source list (i.e., where each group represents multiple rows in a result set).
- *Repeat Alias* – If the Execute SQL action is contained with a Repeat action in your Action Model you may choose its Target alias. In this case, the Context will resolve to a Document and partial XPath to which the Target XPath (see below) will be appended.

When grouping and mapping detail column data, the Declare Group/Repeat, Detail Rows, and Map Target tab work together to define the complete XPath location for the column. (See illustration.) For instance, a column on the Map Target tab will be represented by a Context and XPath. The Context may be a Detail Alias defined on the Detail Rows tab. The Detail Alias in turn will represent another Context and XPath. Its Context may be a Group Alias defined on the Declare Group/Repeat tab. Finally the Group Alias itself will represent another Context and XPath.

By defining the Group and Detail aliases separately, you are able to map rows with duplicate column data (the basis for your groups) just once into group header elements by using the Group alias as a context, and map columns with unique data (the detail of your groups) multiple times within the group header elements by using a Detail Alias whose Context is a Group Alias.

Target XPath: This is an XPath fragment that specifies the custom name to be given to the column and optionally pre-pended by any additional parent elements. The Target XPath will be pre-pended by the Context to produce the final location for the column in the target document.

Base64 encode: The checkbox in this column allows you to convert binary data to an XML-safe representation for use in a DOM element.

Map Target

Columns	Context	Target XPath	Base64 encode
1	gCATEGORY	TheCATEGORY	☐
2	MyOutputDetail	MySKU	
3	MyOutputDetail	MyONHAND	
4	MyOutputDetail	MyCOST	
5			

Detail Rows

Detail Alias:
MyOutputDetail

Representing:
gCATEGORY
PRODUCTDetail

Declare Group/Repeat

Group Alias	Columns	Context	Target XPath
gCATEGORY	CATEGORY	Output	INVENTORYACATEGORYGroup

The three tabs of the Custom Mapping Settings dialog can be used to define sophisticated any-to-any mappings of result-set items to DOM elements. Notice how user-defined aliases (representing, in each case, a DOM context and target XPath) can be substituted back into earlier tab context slots.

A processing summary for the Map Target tab is shown in Table 5–1 below.

Table 5-1

SQL Results	Context = Document	Context = Alias
One Row Returned	One row target is found or created for the first (and only) result row.	One row target is found or created for the first (and only) result row.
Multiple Rows Returned	One row target is found or created for the first result row. Subsequent rows find and map to the <i>same</i> physical target location. (Without an alias, each row’s data is overwritten by the next row until only the last row’s data is left.)	One row target is created for <i>every</i> result row.

Looking at a MapTarget Example

Assuming your Execute SQL action issued the following statement:

```
SELECT CATEGORY, SKU, ONHAND, COST FROM InventorySystem
```

Which returned the following row data:

```
EXECUTED:
SELECT CATEGORY, SKU, ONHAND, COST FROM InventorySystem
```

(See Table 5–2.)

Category	SKU	Onhand	Cost
3	CHR1111	0	999
2	DAD7777	89	245
4	GAR1234	17	100
1	LOR8437	0	275
1	LOR8438	21	375
4	MOM4666	233	300
4	RAC4567	156	230
4	ZAC9080	4	555

Table 5-2

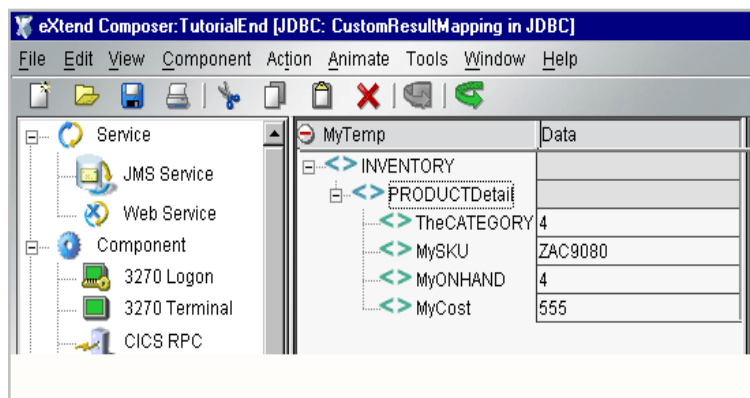
You could fill out the Map Target tab as shown below:

The screenshot shows the 'Custom Mapping Settings' dialog box with the 'Map Target' tab selected. The dialog has three tabs: 'Map Target', 'Detail Rows', and 'Declare Group/Repeat'. The 'Map Target' tab contains a table with four columns: 'Column...', 'Context', 'Target XPath', and 'base64 en...'. There are five rows in the table, numbered 1 to 5. Rows 1 through 4 have the context 'MyTemp' and specific XPath expressions. Row 5 is empty. Below the table is a large empty rectangular area. At the bottom of the dialog are buttons for 'Help', 'OK', and 'Cancel'.

Column...	Context	Target XPath	base64 en...
1	MyTemp	INVENTORY/PRODUCTDetail/TheCATEGORY	☐
2	MyTemp	INVENTORY/PRODUCTDetail/MySKU	☐
3	MyTemp	INVENTORY/PRODUCTDetail/MyONHAND	☐
4	MyTemp	INVENTORY/PRODUCTDetail/MyCOST	☐
5			☐

Column one according to the SELECT statement will be CATEGORY. The Context is a document named “MyTemp” and the target XPath location within the Context will be “INVENTORY/PRODUCTDetail/TheCATEGORY”. Notice that CATEGORY is being renamed to TheCATEGORY and being pre-pended with parent elements of INVENTORY/PRODUCTDetail. This same logic applies to the remaining columns.

However, since we have yet to define or use any aliases, each row’s column data will be written to the same four physical target locations specified on the tab. If only one row is returned, then its data will be mapped to the target document with no problems. If multiple rows are returned as in our example, then each successive row’s data will overwrite the previous row’s data until only the last row’s data exists. (Only in rare cases will this situation be desirable.)



Normally, you use the Map Target tab by itself if only one row is returned and all you wish to do is change the names of the target elements to something different than the column names. (Or if you want to assign different parent elements to individual columns.)

To prevent overwriting data with multiple result set rows you need to use a Detail Alias from the Detail Rows tab telling eXtend Composer to create a new physical target location for each row mapped.

Using The Detail Rows Tab

The Detail Rows tab is used to:

- Create a Detail alias associated with a Context and Target XPath

The Detail Rows tab allows you to create a mapping alias tied to either a document Context or a Group/Repeat alias Context. Use of the Detail Rows tab is optional.

Detail Alias: This is a name you specify that will be referenced as a Context on the Map Target tab for mapping columns in a result set row.

Context: This is a document name or Group/Repeat alias you specify. The Target XPath will be appended to this Context to produce part of the final location for the column in the target document (the remaining part comes from the Target XPath on the Map Target tab). The Context can be a:

- Document – Using a Document name tells eXtend Composer to create a new physical target location once for each row in the result set.
- Group Alias – A Group Alias is defined on the Declare Group/Repeat tab and consists of a Document name and partial Target XPath location. Using a Group Alias tells eXtend Composer to create a new physical target location once for each detail row belonging to each Group (i.e., each group represents multiple rows in a result set).

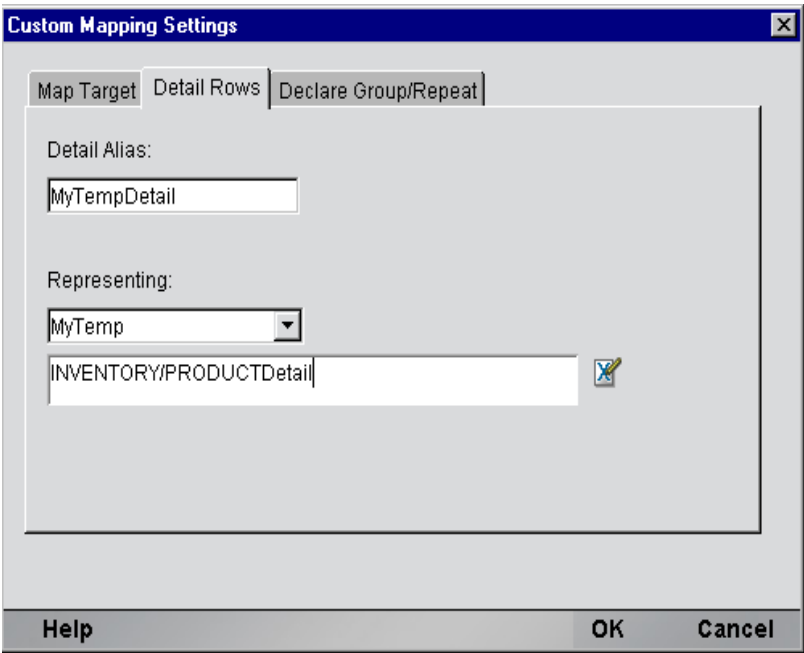
Target XPath: This is an XPath fragment that you specify. It will be pre-pended by the Context on this tab and appended with the Target XPath on the Map Target tab to complete the final location for the column in the target document.

Looking at a Detail Rows Example

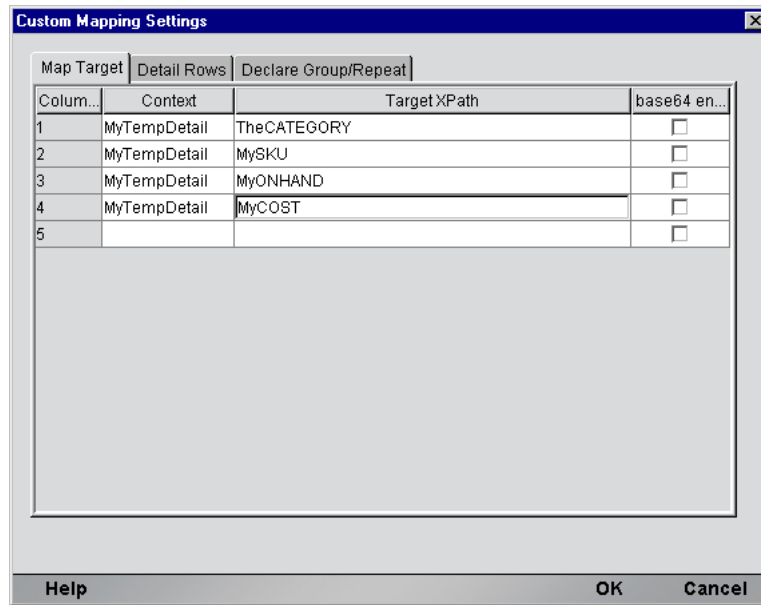
Assuming your Execute SQL action issued the following statement:

```
SELECT CATEGORY, SKU, ONHAND, COST FROM InventorySystem
```

You could fill out the Detail Rows tab as shown below:

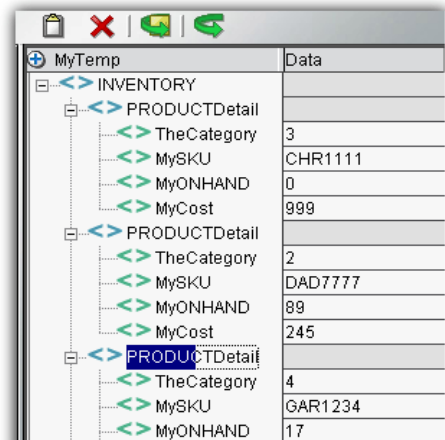


Since the Context MyTemp and Target XPath fragment INVENTORY/PRODUCTDetail are now specified on the Detail Rows tab (creating a new physical target location for each row), references to them must be replaced on the Map Target tab with the Detail Alias “MyTempDetail.” Continuing the example used in the previous section, you would update the Map Target tab as follows:



By using a Detail Alias specified on the Detail Rows tab, you will ensure that if multiple rows are returned in the result set, each row will create a new physical target location under INVENTORY/PRODUCTDetail.

When not used in conjunction with the Declare Group/Repeat tab, you can think of the Detail Rows tab as creating a “Repeat for Row” alias. If the Context for a Column on the Map Target tab is a Detail Alias (instead of a document), then eXtend creates a new Target XPath each time a row mapping occurs. In this way, *multiple rows in the result set create multiple Row Targets in the document without overwriting the previous row’s data*. This is the same functionality provided by the Result Mapping tab’s **Custom...** option, except that you get to rename the columns.



MyTemp	Data
INVENTORY	
PRODUCTDetail	
TheCategory	3
MySKU	CHR1111
MyONHAND	0
MyCost	999
PRODUCTDetail	
TheCategory	2
MySKU	DAD7777
MyONHAND	89
MyCost	245
PRODUCTDetail	
TheCategory	4
MySKU	GAR1234
MyONHAND	17
MyCost	17

However, since we have yet to define or use any Group Aliases, each row's data contains redundant information regarding CATEGORY. Four rows belong to CATEGORY 4, two rows belong to CATEGORY 1, and one row each belong to CATEGORIES 2 and 3. In addition, the rows occur in random order and not together in their groups.

To remove the redundant data from each row and group records together you need to use a Group Alias from the Declare Group/Repeat tab.

Using the Declare Group/Repeat Tab

The Declare Group/Repeat tab is used to:

- Create groups of result set records based on one or more result set columns
- Create a Group Alias to use as a Context for Detail Rows
- Create a Group Alias to use as a Context for Map Targets (creating Group Headers)

By declaring a Group Alias you create a list comprised of the unique values found in a column across multiple rows. Any Map Target column that uses the Group Alias will map its column data only once for each unique Group essentially creating group header information.

In addition, each unique group value points to a list of the rows that belong to it. Any Detail Alias on the Detail Rows tab that uses the Group Alias will map its rows together for that group.

Group Alias: This is a name you specify that is referenced as a Context on the Map Target and/or Detail Rows tabs.

Columns: Specify one or more columns separated by a comma to create your groups. Using two columns means that only unique combinations of the concatenated values of the two columns will create a group.

NOTE The columns you specify must form the basis of an ORDER BY clause in the SELECT statement for the Execute SQL action. If you omit the ORDER BY clause, your results will be unpredictable.

Context: This is a document name in the component or Repeat for Group or Repeat for Element alias in the Action Model that contains the Execute SQL action. The Target XPath is appended to this Context to produce part of the final location for the column in the target document. (The remaining part comes from the Target XPath on the Map Target tab and optionally from the Target XPath on the Detail Rows tab.) The Context can be a:

- Document – Using a Document name tells eXtend Composer to write to the same physical document for each Group.
- Repeat for Group Alias – If your Execute SQL action is inside a Repeat for Group action in your Action Model, then you may use its target alias as the Context for each Group. This tells eXtend Composer to create new Groups once for each Group processed in the enclosing Repeat for Group action.
- Repeat for Element Alias - If your Execute SQL action is inside a Repeat for Element action in your Action Model, then you may use its target alias as the Context for each Group. This tells eXtend Composer to create new Groups once for each repeating element processed in the enclosing Repeat for Element action.

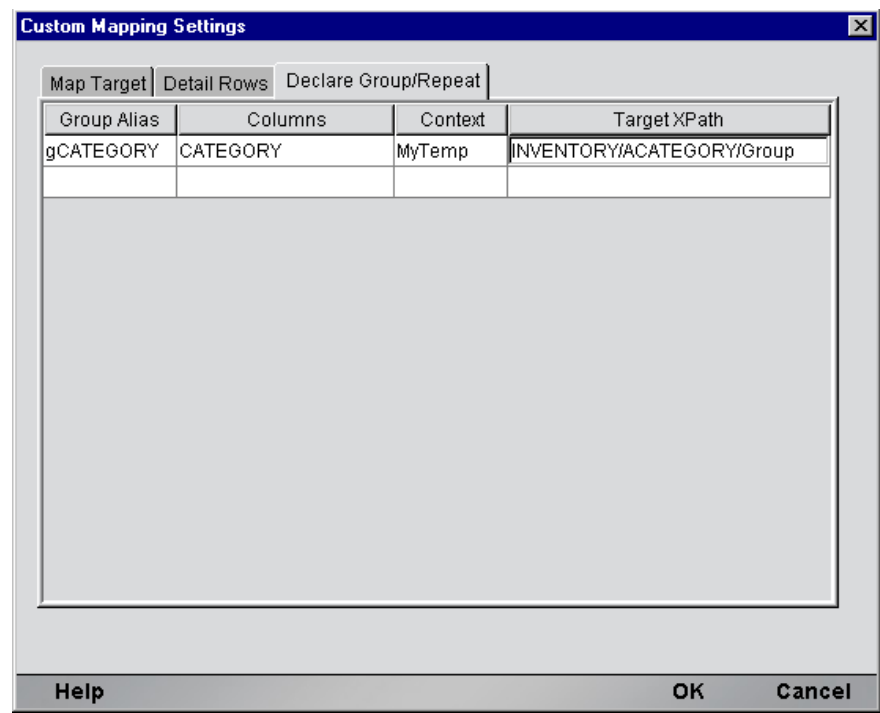
Target XPath: This is an XPath fragment that you specify. It is pre-pended by the Context on this tab and appended with the Target XPath on the Map Target tab (and optionally with the Target XPath on the Detail Rows tab) to complete the final location for the column in the target document.

Looking at a Declare Group/Repeat Example

Assuming your Execute SQL action issued the following statement:

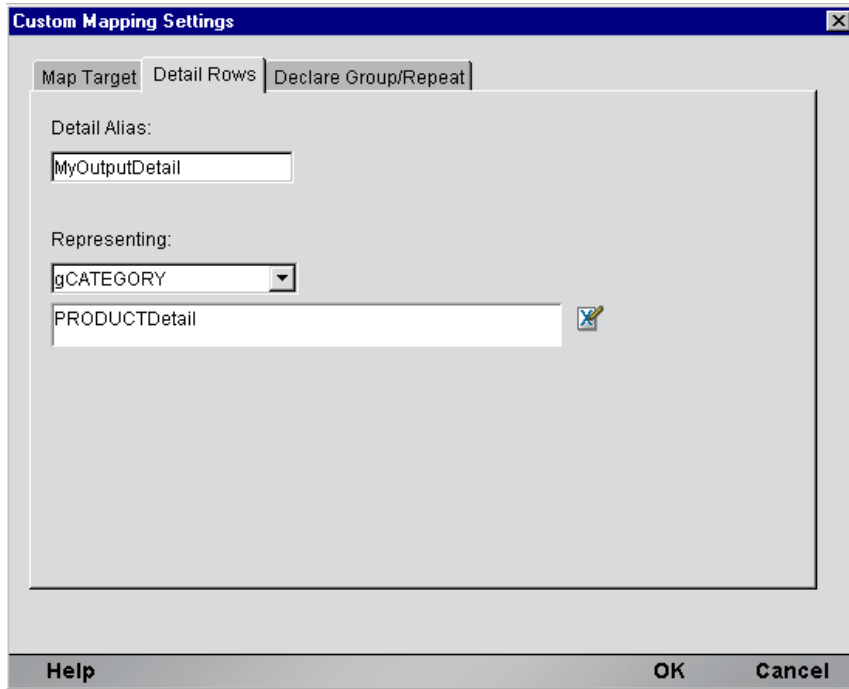
```
SELECT CATEGORY, SKU, ONHAND, COST FROM InventorySystem order by CATEGORY
```

You could fill out the Detail Rows tab as shown below:



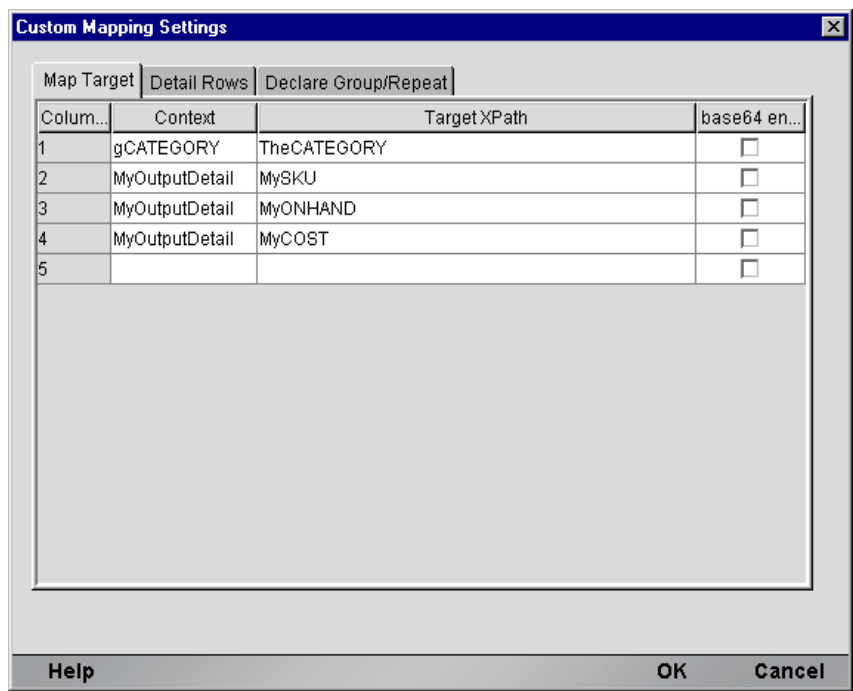
Similar to the example for Detail Rows, since the Context MyTemp (and Target XPath fragment INVENTORY/PRODUCT) is now specified on the Declare Group/Repeat tab, references to it must be replaced on the Detail Rows tab with the Group Alias “gCATEGORY.” In addition, you are no longer listing just PRODUCTDetail under INVENTORY but rather groups of PRODUCTDetail so a new element is introduced into the Group’s Target XPath called “ACATEGORYGroup.” Thus for each Group mapped, a new ACATEGORYGroup element is created.

Continuing the example used in the previous two sections, you would update the Detail Rows tab as follows:



Notice that the Context of “MyTemp” has been replaced by the Group Alias gCATEGORY which represents MyTemp/INVENTORY/ACATEGORYGroup. This means that Detail Rows belonging to the Group are the only ones mapped, instead of all the Detail Rows.

Continuing the example used in the previous two sections, you would update the Map Target tab as follows:



We have replaced the Context for the CATEGORY column with the Group Alias. This means that CATEGORY is only mapped once for each Group instead of once for each detail row.

The screenshot shows a software window titled "JDBC: CustomResultMapping in JDBC". It has a menu bar with "Action", "Animate", "Tools", "Window", and "Help". Below the menu is a toolbar with icons for a folder, a document, a red X, a green checkmark, and a green arrow. The main area is divided into two panes. The left pane shows a hierarchical tree structure for a mapping named "MyTemp". The tree starts with "MyTemp" (expanded), which contains "INVENTORY" (expanded). "INVENTORY" contains three "ACATEGORYGroup" elements (expanded). Each "ACATEGORYGroup" contains "TheCATEGORY" and "PRODUCTDetail" elements (expanded). "TheCATEGORY" elements have values 3, 2, and 4. "PRODUCTDetail" elements have sub-elements "MySKU", "MyONHAND", and "MyCost" with values like "CHR1111", "0", "999", "DAD7777", "89", and "245". The right pane is a table with a header "Data" and rows corresponding to the data in the tree.

	Data
MyTemp	
INVENTORY	
ACATEGORYGroup	
TheCATEGORY 3	
PRODUCTDetail	
MySKU	CHR1111
MyONHAND	0
MyCost	999
ACATEGORYGroup	
TheCATEGORY 2	
PRODUCTDetail	
MySKU	DAD7777
MyONHAND	89
MyCost	245
ACATEGORYGroup	
TheCATEGORY 4	

When you declare a Group Alias, the result set rows are scanned and organized into groups establishing how many processing loops will occur during mapping. If eight rows are in the result set with only four different values (e.g., 3, 2, 4, 1, 1, 4, 4, 4) then there will be four group mapping loops (e.g., 1, 2, 3, 4) and eight detail loops tied to their appropriate group mapping loops (e.g., group one has its two detail rows, group two has its one detail row, group three has its one detail row, and group four has its four detail rows).

Using the prior graphics, you can trace how the final context for the Map Target columns is constructed for Column one and Column two. Column one is the CATEGORY from the result set. Its name in the DOM will be TheCATEGORY. Its ancestor elements are determined by the context "gCATEGORY" defined as MyTemp/INVENTORY/ACATEGORYGroup on the Declare Group/Repeat tab. So the final XPath for CATEGORY is: Output/INVENTORY/ACATEGORYGroup/TheCATEGORY. Since the context for TheCATEGORY is a Group alias, it will be mapped once for each group or four times as determined earlier.

Column two is the SKU data from the result set. Its name in the DOM will be MySKU. Its ancestor elements are determined by the context “MyTempDetail” defined to be gCATEGORY (defined above) plus PRODUCTDetail. So the final context for the column will be MyTemp/INVENTORY/ACATEGORYGroup/PRODUCTDetail/MySKU. Since the context for MySKU is a Detail Alias, it is mapped once for each Detail Row. However, each Detail Row has a Context of a Group Alias limiting mapping to only those detail rows that belong to the Group.

6 Stored Procedure Mapping

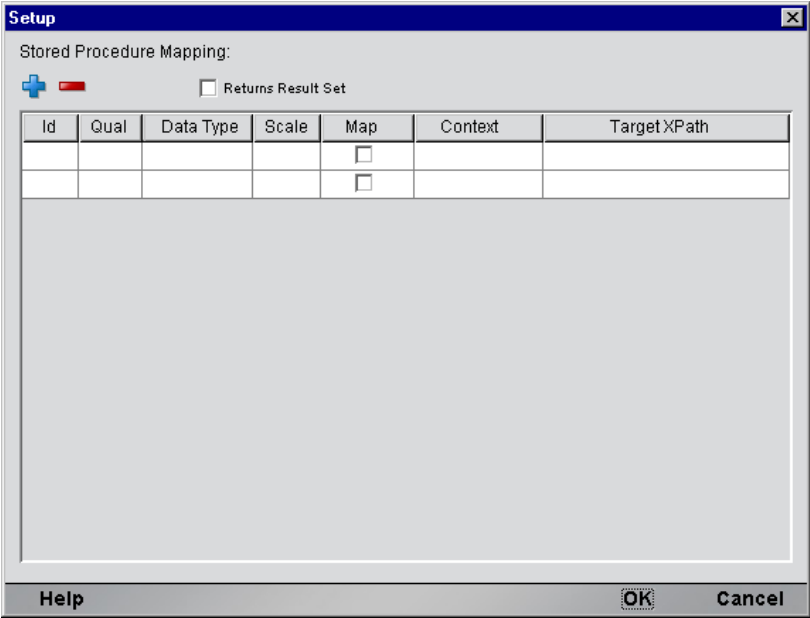
eXtend Composer supports the mapping of data returned by stored procedures. The following sections describes the stored procedure mapping features.

About Stored Procedure Mapping

eXtend Composer allows for mapping the data returned by stored procedures to DOM/Node combinations. To do so, select the **Stored Procedure mapping** option on the **Result Mapping** tab in the **Query/Results Mapping** Pane.

The screenshot shows the 'Result Mapping' tab of the 'Query/Results Mapping' pane. It has three tabs: 'SQL Statement', 'Result Mapping', and 'Result Text'. The 'Result Mapping' tab is active. Inside, there's a 'Result Row Placement' section with a label 'Enter the XML element to place results under:'. Below this is a dropdown menu showing 'Temp' and a text input field containing 'INVENTORYSTATUSROW'. To the right of the text field is a small icon of a document with a pencil. Below the 'Result Row Placement' section are four checkboxes: 'Create element names as column names' (checked), 'Create elements if column is null' (checked), 'Include datatype info in element attribute' (unchecked), and 'Generate Row numbers' (unchecked). Below these checkboxes are two radio buttons: 'Custom Column/Row/Group' (selected) and 'Stored Procedure mapping' (unchecked). To the right of the 'Custom Column/Row/Group' radio button is a 'Custom...' button. To the right of the 'Stored Procedure mapping' radio button is a 'Setup...' button.

This will enable the **Setup...** button. Press the **Setup...** button to display the **Setup** dialog for **Stored Procedure Mapping**.



Binding Rules

It is important to understand that eXtend Composer binds to all **Expressions** as well as placeholders represented by **Question Marks** (e.g. either ‘:<expression>’ or ?), but not to Constants (e.g. ‘abc’).

Using the Stored Procedure Mapping Setup Dialog

The Stored Procedure Mapping Setup dialog is used to map the data returned by a stored procedure. The Setup dialog allows you to specify Context/Target XPath combinations for the returned data.

Use the + and - controls to add and delete Context /Target XPath combinations.

Oracle RDBMSs return result sets as parameters. Non-Oracle RDBMSs return result sets, but not as parameters. Select the **Returns Result Set** checkbox when result sets are returned by non-Oracle RDBMSs. Selecting the **Returns Result Set** checkbox for non-Oracle RDBMSs enable eXtend Composer to find the returned result set.

NOTE All Expressions and placeholders (e.g.: '?') must be specified in the Stored Procedure Mapping Setup dialog in order to correctly map the returned data.

For each returned Input/Output parameter (which may be expressions) and each Output parameter (which may be either an expression or a ?) (see the **Rules for Stored Procedures** section in Chapter 4), complete the following:

Id: Based on the SQL parameters, Id is the one-based number sequence of the return values you're expecting. Using **Id**, you will need to explicitly specify the *sequence positions* of each of the parameters containing either expressions (e.g. '<ExpressionName>') or placeholders (e.g. '?'). For example, the following procedure call has three parameters: a constant, 'Process', a placeholder, '?', and an expression, ':Smith'. The value 'Process' does not need an Id in the **Stored Procedure Mapping** pane since eXtend Composer does not bind to values. The Id entries for the placeholder - '?' and the variable ':Smith' are, respectively, 2 and 3. eXtend Composer binds to variables and placeholders, therefore, they must be specified in the **Stored Procedure Mapping** pane in order to properly map the data returned by a stored procedure.

Example:

```
{ call DemoPackage.sp_withParams('Process', '?', ':Smith') }
```

Qual: Qual qualifies the parameter as an Input parameter, an Output parameter or as an Input/Output parameter.

Data Type: Data Type is a drop down list which provides the following options: VARCHAR, DECIMAL, DATE, BINARY or Oracle Result Set. When **Oracle Result Set** is selected, Context and Target XPath do not apply (N/A) and are, therefore, disabled.

Scale: The value of Scale specifies the decimal place precision.

Map: The Map checkbox is selected to map the parameter.

Context: this specifies the target document for the column. The Target XPath will be appended to the Context to produce the full XPath location for the column in the target document. The Context can be a:

- *Document* – You may use this choice if your result set contains only one row, otherwise each additional row will overwrite the previous row's data.
- *Detail Alias* – A Detail Alias is defined on the Detail Rows tab and consists of a Document name and partial Target XPath. Or the Detail Alias may consist of a Group Alias (defined on the Declare Group/Repeat tab) and partial Target XPath location. Using a Detail Alias tells eXtend Composer to create a new physical target location for each member of the source list (i.e., each row in a result set).
- *Group Alias* – A Group Alias is defined on the Declare Group/Repeat tab and consists of a Document name and partial XPath location. Using a Group Alias tells eXtend Composer

to create a new physical target location once for each Group in the source list (i.e., where each group represents multiple rows in a result set).

- *Repeat Alias* – If the Execute SQL action is contained with a Repeat action in your Action Model you may choose its Target alias. In this case, the Context will resolve to a Document and partial XPath to which the Target XPath (see below) will be appended.
- *-- via standard --* will use the **Result Mapping** tab's Result Row Placement specification.
- *-- via custom --* will use the settings on the **Custom Mapping Settings** dialog.

When grouping and mapping detail column data, the Declare Group/Repeat, Detail Rows, and Map Target tab work together to define the complete XPath location for the column. (See illustration.) For instance, a column on the Map Target tab will be represented by a Context and XPath. The Context may be a Detail Alias defined on the Detail Rows tab. The Detail Alias in turn will represent another Context and XPath. Its Context may be a Group Alias defined on the Declare Group/Repeat tab. Finally the Group Alias itself will represent another Context and XPath.

By defining the Group and Detail aliases separately, you are able to map rows with duplicate column data (the basis for your groups) just once into group header elements by using the Group alias as a context, and map columns with unique data (the detail of your groups) multiple times within the group header elements by using a Detail Alias whose Context is a Group Alias.

Target XPath: This is an XPath fragment which will be appended to Context to specify the full XPath location into the target document.

Returned Result Set

A result set is mapped to a document with elements created from the result set's column names.

- Target element names created in the document are the same as column names returned in the result set
- All columns returned in the result set are mapped to the target document
- All columns are mapped to the same parent target element
- All rows are placed into a single document

NOTE Any column names that contain spaces will have the spaces replaced with an underscore character since XML does not permit spaces in element names.

A JDBC Glossary

Connection Pool

A set of database connections managed by the application server for the various applications it manages.

Custom Result Mapping

The Custom Result Mapping dialog provides a similar alias ability for repeating rows in SQL result sets as the Declare Group and Repeat for Group actions do for repeating elements in a document.

Declare Group/Repeat Tab

This tab of the Custom Results Mapping dialog is used to create groups of result set records on one or more result set columns, create a Group Alias to use as a Context for Detail Rows, and create a Group Alias to use as a Context for Map Targets (creating Group Headers).

Detail Rows Tab

This tab of the Custom Results Mapping dialog allows you to create a mapping alias tied to either a document Context or a Group/Repeat alias Context. Use of the Detail Rows tab is optional.

DOM

A Document Object Model (DOM) is an XML document constructed as an object in a software program's memory. It provides standard methods for manipulating the object. In Composer, DOM is often synonymous with XML Document. DOMs are represented as hierarchical trees with a single root node.

DOM Context

The name of a DOM (Input, Output, Temp, etc.), or the name of a Repeat alias previously defined in the component. (The alias itself represents a DOM context, representing the nodepath hierarchy upstream of a given element.)

Execute SQL Action

Same as SQL Statement Action.

JDBC

A Sun trademark for the Java API for accessing relational database data. It is commonly assumed to mean Java Database Connectivity.

Map Target Tab

This tab of the Custom Results Mapping dialog is used to create target element names for each result set column and specify a target Context for each result set column.

Native Environment Pane

A pane in the JDBC Component Editor that simulates an actual SQL environment when you issue a query.

Query/Result Mapping Pane

(Same as the Native Environment Pane.) A pane in the JDBC Component Editor that includes three tabs: the SQL Statement tab, the Result Mapping tab, and the Results Text tab.

Result Mapping Tab

A tab in the Query/Result Mapping Pane that allows you to map the result of your database query to an XML document.

Result Text Tab

A tab in the Query/Result Mapping Pane that displays the actual data that was returned following the execution of the database query.

Row Target

The receiving element in a mapping operation is called the *row target*. It represents a specific position in the DOM tree of an XML file.

SQL Statement Action

Most commonly used to query an existing database and then map the result to an XML document.

SQL Statement Tab

A tab in the Query/Result Mapping Pane that allows you to write or build SQL commands.

SQLCODE

A global ECMAScript variable created by the execution of SQL statements. Contains a status code generated by the database engine.

SQLSTATE

A global ECMAScript variable created by the execution of SQL statements. Contains information generated by the database engine.

UPDATECOUNT

A global ECMAScript variable created by the execution of SQL statements. Contains a count of the number of rows changed by the database engine.

B Reserved Words

The following terms are reserved words in eXtend Composer for the JDBC Connect and should be avoided in any user created labels or objects.

- SQLCODE
- SQLSTATE
- UPDATECOUNT
- LASTSQL

Index

A

- action menu 40
- action model 25
- actions
 - overview 25
 - using basic and advanced 40
- Advanced Action 42
- advanced actions 40
- alias
 - and custom result mapping 47
- Allow SQL Transactions 17
- auto-commit 17

B

- base64 encode 48
- Basic Actions 40
- basic actions 40
- batch actions (see SQL Batch) 36

C

- code table map, creating 18
- colons, special meaning in SQL action 33
- commit 17
- component
 - creating new 19
- component editor window 22
- connection

- creating 13
 - dirty 17
- connection pool 14
 - definition of 67
- Constant and Expression Driven Connections 13
- context 47, 52, 56
- Custom Mapping Settings 66
- custom result mapping 46, 47
 - definition of 67
- custom script
 - creating 18

D

- Data Exchange Actions 42
- Data Type 65
- database-specific parameters 16
- DB Params 16
- declare group/repeat example 57
- Declare Group/Repeat tab 56
 - definition of 67
- default result mapping 45
- detail alias
 - creating 52
 - used as a context 48, 65
- detail rows example 53
- Detail Rows tab 52
 - definition of 67
- Discard Batch 37
- document, used as a context 48, 65

E

- ECMAScript
 - in SQL Statements 33
- ECMAScript functions, using 44
- errors and SQL messages 44
- example query 28
- Execute as Prepared 25, 26
- Execute Batch 37

Execute SQL action
 definition of 67
executing the SQL statement 33
Expressions 64

G

group alias
 creating 56
 used as a context 48, 65

H

Help 11

I

Id 65

J

JDBC
 creating XML templates for 18
 definition of 67
 overview 8
 what does it do 9
JDBC component
 about 10
 creating new 19
JDBC Component Editor
 about the window 22
 building applications 11
JDBC connection pools 14
JDBC connection resource 13
JDBC drivers 14

L

LASTSQL 44

M

map target
 example 50
Map Target tab 47
 definition of 67

N

native environment pane
 definition of 68

O

Oracle Result Set 65

P

precompiled SQL 25
prepared SQL statements 25

Q

Qual 65
query, building an example 28
Query/Result mapping pane 22
 definition of 68
Query/Result Mapping Pane. 26

R

Repeat Actions 43
repeat alias

- creating 56
- used as a context 48, 66
- Result Mapping 66
- result mapping
 - using custom 46
 - using default 45
- result mapping tab 23
 - definition of 68
- result text tab 24
 - definition of 68
- rollback 17
- row target 45, 46

S

- S3SqlAnywhereAuth 17
- Scale 65
- scope of SQL batches 38
- SQL
 - prepared statements 25
 - transaction verbs 17
- SQL Anywhere 16
- SQL Batch Action 36
- SQL messages 44
- SQL statement
 - building 27
 - checking the results 34
 - executing 33
- SQL statement action
 - definition of 68
- SQL statement tab 23
 - definition of 68
- SQLCODE 44
 - definition of 68
- SQLSTATE 44
 - definition of 68
- Start Batch 37
- Stored Procedure Mapping 63

T

- target element names 47
- target XPath 47, 48, 52, 56, 66
- transactions
 - auto-commit flag 17
 - SQL 17
- Try/On Error 37

U

- UPDATECOUNT 44
 - definition of 68

X

- XML template
 - creating 18

